الموضوع: _____ التاريخ: / /

«Data Structures and Algorithms»

«هياكل البيانات والخوارزميات»

هياكل البيانات والخوارزميات ←
① هيكل يحتوي على بيانات
② مخزونة في الذاكرة
③ بطريقة منظمة داخل هيكل بياني

الخوارزميات ← هي عملية تنظيم البيانات داخل هياكل بيانية منظمة في الذاكرة من أجل خزنها واسترجاعها في وقت احترفي طريقة كتابة برامج كفوءة للتعامل مع هذه البيانات المخزونة في الهيكل البياني

هياكل البيانات ← هو هيكل يحتوي على بيانات تكون مخزونة في الذاكرة بطريقة منظمة داخل هياكل بيانية والخوارزميات هي الطريقة التي يتم التعامل بها مع البيانات المخزونة في الهيكل البياني

* ١- ابدء
٢- احضر عدد
٣- اجمع العدد
٤- اطبع

* ابسط الحالي تكون جملة واحدة اما اكثر عدد وكلتي بيها

* واجب عملي:-
قراءة جملة اما اكثر

مفهوم الهيكل البياني

```
int A[10];
cout
cin
```

الموضوع: _____ التاريخ: / /

Lec 1 :-

"تعريف مهم"

هياكل البيانات :- هو دراسة طرق جمع وتمثيل البيانات في ذاكرة الحاسوب حتى طريقة يتاح هياكل بيانية تدعم نوع البيانات المراد تخزينها بحيث تمكن التخزين من الوصول لهذه البيانات والتعامل معها بكفاءة عند اجراء عمليات مثل الاضافة والحذف والتحديث

* الخوارزميات هي البرامج التي مسؤولة عن إدارة البيانات الموجودة في الهياكل البيانات

* وهذه البيانات تترتيب بعلاقات محددة تختلف العلاقات التي تربط البيانات الموجودة في هياكل بيانية اخرى

* تقسم هياكل البيانات أو تمثيل البيانات او هيكل البيانات الى نوعين Data Representation Typs طرق تمثيل هياكل البيانات او البيانات

① Logical structure هيكل منطقي

② physical structure هيكل فيزيائي

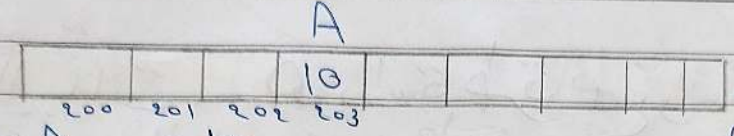
① Logical / Representation structure هو تمثيل مفهوم من قبل المبرمج ويكون تمثيله على شاشة الحاسوب

* Ex: int A=10 و

② physical / Representation structure هو تمثيل بياناتي مفهوم من قبل المعالج وتعامل مع الذاكرة مباشرة

التاريخ: / /

الموضوع:



- Cout << A → 10

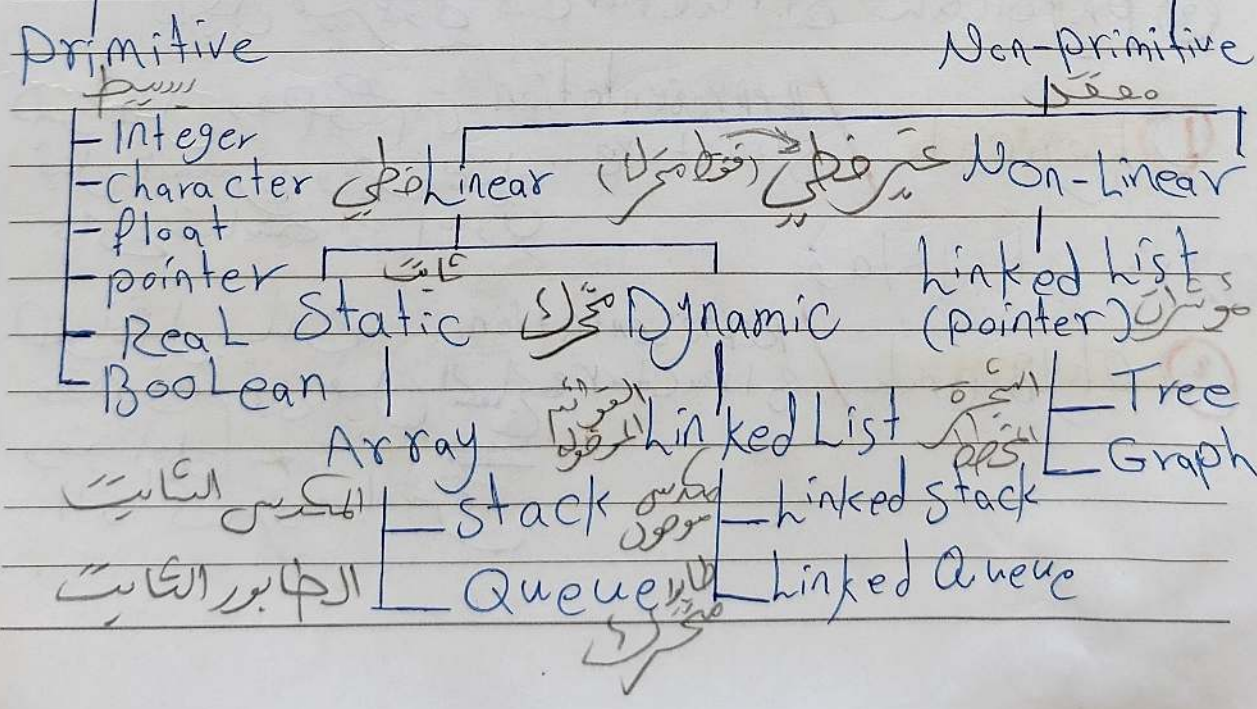
⊗ استخدم A للدلالة على logical

⊗ التمثيل الفيزيائي هو عبارة عن موقع ذاكرة تحتوي على البيانات اما التمثيل المنطقي والاتجاهات التي يكتبها المبرمج هو كل تمثيل وفكرة معينة

⊗ طالما اكو موقع وعنوان ذاكرة من اسمه ذاكرة يشير بالذاكرة هذا تمثيل فيزيائي اما شي تفكر اجته هذا تمثيل منطقي

⊗ خطط هيكل مهم

Data Structures Diagram:
خطط هياكل البيانات



الموضوع: _____ التاريخ: / /

المؤشرات ← *A ← Cout <<

* ملاحظة مهمة - أو سؤال
- ما هي العوامل التي تحدد كفاءة الهيكل البياني؟

1) Speed السرعة عند التعامل مع البيانات مثل إضافة تعديل
2) Memory توفير المساحة التخزينية بمعنى كلما كان الهيكل البياني يوفر مساحة تخزينية كلما كان هناك هيكل بياني كفؤ

كيفية اختيار الهيكل البياني المناسب:-
* عوامل تحدد الاختيار
1) حجم البيانات (إذا كان حجم البيانات معروف عادة نختار هيكل بياني حجمه يناسب هذه البيانات هو الثابت) ويقصد به المساحة التخزينية المطلوبة لتمثيل الهيكل البياني
2) سرعة وطريقة استخدام البيانات بمعنى الزمن اللازم لمعالجة أي معلومة داخل الهيكل البياني (إضافة - إضافة - تعديل)
3) أسلوب البرمجة

1) حجم 2) سرعة 3) أسلوب برمجة

Lec 2:-
هياكل البيانات الخطية Linear structures
- هو عبارة عن تتابع هياكل البيانات الذي يعتمد مبدأ التسلسل في تخزين عناصر الهيكل البياني في الذاكرة بمعنى العنصر الأول يتبعه العنصر الثاني والثاني يتبعه العنصر الثالث وهكذا بقية العناصر
* أهم مبدأ في هياكل البيانات هو خطي بمعنى تسلسلي

الموضوع: _____ التاريخ: / /

* تقسم هياكل البيانات الخطية الى قسمين:
1. هياكل البيانات الخطية الثابتة Static linear structure
2. هياكل البيانات الخطية المتحركة Dynamic linear structure

* الثابت والمتحرك هم اتنين خطية يعني بمعنى يشتركون في شغلة واحدة وهي مبدأ التسلسل لخزن البيانات في الذاكرة

* الاختلاف بينهم: الثابت يستخدم مواقع خزنية متعاقبة او متجاورة (غير كل) اما المتحركة لا تحتمل شرط التجاور ممكن ان تكون متجاورة او غير متجاورة

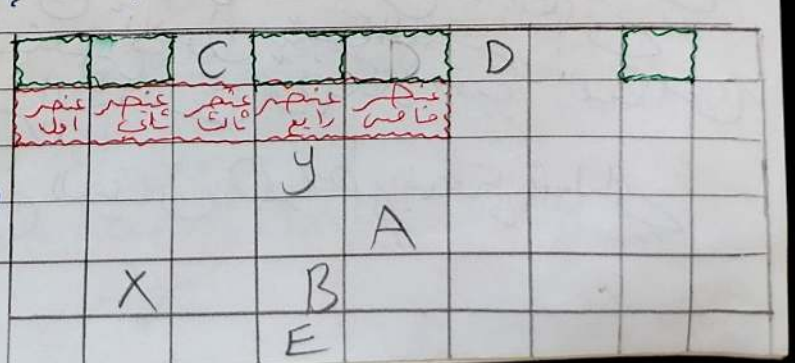
* تشترك هياكل البيانات الخطية الثابتة مع هياكل بياناتي الخطية المتحركة بمبدأ التسلسل في خزن البيانات

* تختلف هياكل البيانات الخطية الثابتة عن هياكل البيانات الخطية المتحركة في عملية خزن البيانات من حيث التعاقب او التجاور

* يتم الربط بين عناصر هذه البيانات الخطية المتحركة عن طريق استخدام المؤشرات

* الفرق الجوهري بينهم: الثابتة اكثر بكون متعاقبة او متجاورة اما الخطية متحركة ليس بالضرورة ان يكون متعاقبة او متجاورة

- تمثيل هيكل بياني خطي ثابت متعاقبة كما في الصورة (A)
- تمثيل هيكل بياني خطي متحرك متجاورة كما في الصورة (B)



الموضوع: _____ التاريخ: / /

Array: هو أحد أنواع هياكل البيانات الخطية من النوع الثابت يعتمد مبدأ التسلسل والتعاقب او التجاور في خزن البيانات في الذاكرة ويتكون من العناصر البيانية

① جميع العناصر البيانية تكون من نوع واحد int, float
② يمكن الوصول عشوائيا دون الاعتماد على اي موقع اخر في نفس المصفوفة
③ وقت وصول متساوي لجميع عناصر المصفوفة (من خاصية الوصول العشوائي)
④ مواقع العناصر تبقى ثابتة ولا تتغير اثناء التعامل مع اي من عناصر المصفوفة
⑤ تمثل للمصفوفة باستخدام مواقع متعاقبة في الذاكرة
⑥ المصفوفة لا يمكن اجراء اي تعديل عليها بعد تنفيذ البرنامج لانها هيكل بياني ثابت

cout<<a[4];

سؤال الكوز: hec 2 - اشرح بالتفصيل التمثيل المنطقي والفيزيائي للبيانات مع ذكر أمثلة - توضيحية؟

Logical

① وجهة نظر المبرمج للبيانات
② تمثيلة على الشاشة
③ التعامل مع لغات البرمجة ذات المستوى العالي
④ int A=0;

Physical

① وجهة نظر العتاد للبيانات
② تمثيلة في الذاكرة
③ التعامل مع لغات البرمجة ذات المستوى الواطئ
⑤ int A 10 — 500 في الذاكرة

التاريخ: / /

الموضوع: _____

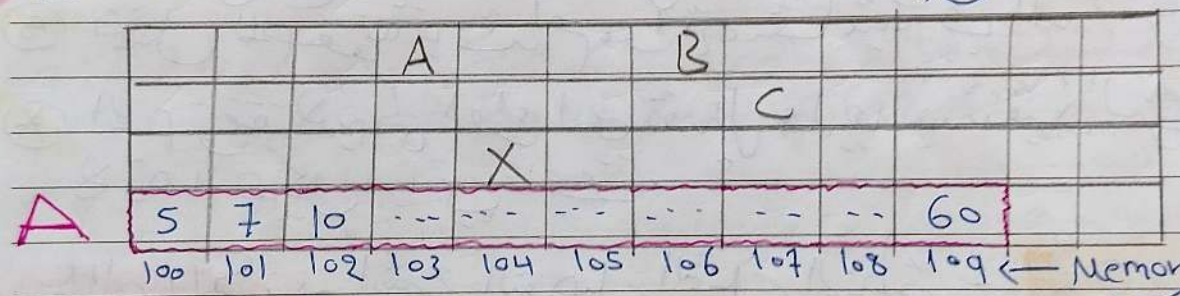
Array :- ① 1D Array Representation

(A) - Logical Representation تمثيل منطقي

Ex: int A[10]; ⟶ A[1] = 5;
A[2] = 7;
A[3] = 10;
⋮
A[10] = 60;

Cout << A[3]
ترجيلي بـ "قيمة داخل عنصر ثالث"
10

(B) - Physical Representation تمثيل فيزيائي



* يستخدم المعالج معادلة للوصول الى موقع ذاكرة معين والعنصر المطلوب
address عنوان الذاكرة

* اذا اشتغل C++ أشيل الواحد

Location A[i] = BA + (i - 1)

- Location A[i]: الموقع المطلوب
- BA: Base address عنوان ذاكرة اول عنصر في المصفوفة
- (i - 1): الموقع المطلوب

ex:- Location A[3] = 100 + (3 - 1)
= 102

التاريخ: / /

الموضوع:

Ex: Comput the memory address of A[5] ; if A size equal 10 and BA = 1500

* هنا الـ A[5] بيساوي 10 وليس أدرس على والموقع كمان

Sol: location A[i] = BA + (i-1)

A[5] = 1500 + (5-1)

= 1504 memory address

* اذا طلب الدكتور أشرح طريقة تمثيل الحواسيب العادية الذاكرة هنا لازم اكتب التعريف بالمنطقي واشرح شلون بتم تمثيلها

كي: اشرح طريقة تمثيل المصفوفة الاحادية في الذاكرة

الحل ممكن بطريقتين الطريقة الاولى والثانية

- كتديد مواقع متجاورة في الذاكرة مثال A[10] وأرسم

(A)

A	B	X	Y	...	Z
400	401	...	...	...	...

* طريقة تكون للعناصر تقع بشكل واحد

للوصول الى اي عنصر مثال هذه المصفوفة تحتاج هذا القانون

Location .....

Ex:

(2) 2D Array

(A) - Column method

طريقة الاعمدة

* هاي الطريقة معناها الحاسوب يأخذ العمود الاول يخلي بالذاكرة بعدين يجي ياخذ العمود الثاني مباشرة بعد الاخر عنصر من عناصر العمود الاول وهكذا عمود وراء عمود.

تمثيل منطقي

15	20	30	5
a_{11}	a_{12}	a_{13}	a_{14}
6	7	10	8
a_{21}	a_{22}	a_{23}	a_{24}
12	31	22	33
a_{31}	a_{32}	a_{33}	a_{34}

cout << A[2][3];

↓

10

* الشكل الاسفل أصبح 12 موقع متجاور في الذاكرة لان فيه صفوف و4 اعمدة

• 3×4=12

التاريخ: / /

الموضوع:

(A) - Column method طريقة الأعمدة

((عمود ورا عمود)) العمود تلو العمود

BA ← 100 العمود الأول 102 العمود الثاني 103 104 105 ---- 107

a_{11}	a_{21}	a_{31}	a_{12}	a_{22}	a_{32}	a_{13}	a_{23}	a_{33}	a_{14}
a_{24}	a_{34}								

العمود (A)

$$\text{Location } A[i][j] = BA + M \times (j-1) + (i-1)$$

- BA: عنوان الذاكرة أول عنصر في المصفوفة
- M: عدد الصفوف الكلي
- (j-1): العمود الحالي
- (i-1): الصف الحالي

* بطريقة الأعمدة نبدأ بعد الصفوف الكلي أما بالطريقة الثانية نبدأ بعد الأعمدة الكلي يعني عكس التمرين

Ex: Let A[3][4], BA = 100 Compute Address (M = 3, N = 4)

Sol: Location A[2][3] = 100 + 3 × (3-1) + (2-1)

= 100 + 6 + 1

= 107 memory address

A[i][j] (i: الأسطر، j: الأعمدة)

* الأولوية بالعمليات هي كما تأتي القوس بعدين

(B) - Row method طريقة الصفوف

* هذه الطريقة تعني الأولى تقريباً أخذ الصفوف وتخزينها في مواقع متتالية أو متجاورة في الذاكرة ثم نأخذ عناصر الصف الثاني ونضعها بعد آخر عنصر من عناصر الصف الأول وهكذا بقية الصفوف.

الموضوع: _____ التاريخ: / /

(B) Row method طريقة الصفوف

$$\text{Location } A[i][j] = BA + N*(i-1) + (j-1)$$

- N: عدد الأعمدة الكلي
- $(i-1)$: عدد الصفوف الطولي
- $(j-1)$: عدد العناصر العرضي

Let $A[\overset{M}{10}][\overset{N}{15}]$, $BA = 2000$

Ex: Compute memory address / $A[\overset{i}{3}][\overset{j}{4}]$ using row method

$$\text{Location } A[3][4] = 2000 + 15*(3-1) + (4-1)$$
$$= 2000 + 30 + 3$$
$$= 2033 \text{ memory Address}$$

س: اشرح كيفية تمثيل المصفوفة الثنائية في الذاكرة باستخدام طريقة الصفوف (أو الأعمدة)

تؤخذ جميع عناصر الصف الأول للمصفوفة الثنائية وتخزن في مواقع خزنية متجاورة في الذاكرة وبعد آخر عنصر من الصف الأول يتم ادخال عناصر الصف الثاني وهكذا مع بقية الصفوف مع مثال:

* موجود بالمذكرة

صفحة 4

مع الرسم

الموضوع: _____ التاريخ: / /

Lec3

سؤال الكويز:

Q: Given Array A[15][20], Base Address = 2000 compute the memory address of A[10][9] using Row method

باستخدام طريقة الصفوف

Sol:-

$$\text{Location } A[i][j] = Ba + N * (i-1) + (j-1)$$

$$\text{Location } A[10][9] = 2000 + 20 * (10-1) + (9-1)$$

$$= 2000 + 180 + 8$$

$$= 2188 \text{ memory address}$$

Stack Using Array:

المكدس باستخدام المصفوفة

مكدس ثابت Static Stack:

هو هيكل بياني خطي من النوع الثابت (يستخدم المصفوفة) يعمل وفق مبدأ من يدخل اولاً يخرج اخراً والعكس صحيح

Last In first Out (LIFO)

first In Last Out (FILO)

LIFO:- وهي تمثل مبدأ عمل المكدس بمعنى ان اخر عنصر يدخل الى المكدس هو اول عنصر يخرج منه.

المكدس بصورة عامة:- هيكل بياني يتم تمثيله في ذاكرة الحاسوب يعمل وفق مبدأ من يدخل اولاً يخرج أخراً ويتم استخدام مؤشر (top) للدلالة على أخر عنصر تم اضافته الى المكدس لا يشير الى قمة المكدس)).

الموضوع: _____ التاريخ: / /

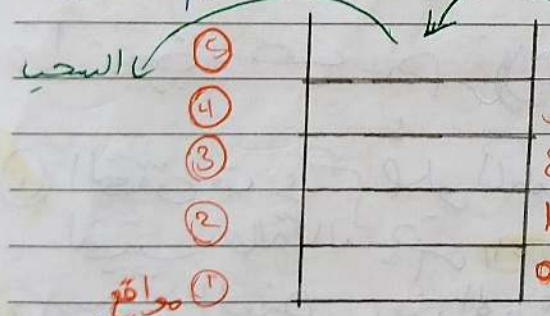
لأجل تمثيل المكدس في الذاكرة باستخدام المصفوفة يتم حجز مصفوفة بالحجم المطلوب (Size) واستخدام ومؤشر إضافة (top) يشير الى قمة المكدس (آخر عنصر تم ادخاله الى المكدس) قيمة المؤشر top قبل أضافة اي عنصر الى المكدس تساوي (-1) بمعنى ان المكدس خالي من العناصر

العمليات التي يمكن اجراءها على المكدس:

① أضافة عنصر (Push)
② سحب عنصر (Pop)
③ التحقق من إمتلاء المكدس بالعناصر (is full)
④ " " خلو المكدس من العناصر (is Empty)

* لبيان ان المكدس ثابت حجمه ثابت او معرض لعملية الامتلاء

stack Size = 5 حجم المكدس



خوارزمية أضافة عنصر الى المكدس:

① التحقق من عدم أمتلاء المكدس لتجنب حالة الفيض (Over flow) عندما (top == Size - 1)
* أكتب قاعدة امتلاء المكدس الثابت؟ top == Size - 1
② تحديث قيمة المؤشر top يشير الى الموقع الجديد top = top + 1
③ أضافة العنصر الجديد Stack[top] = newitem
العنصر الجديد
* من المؤشر يكون عالي قمة المكدس هنا المكدس ممتلئ ويتعذر عملية الاضافة ويعرض رسالة (Over flow)

الموضوع: _____ التاريخ: Size=5

5		S[4]
4		S[3]
3		S[2]
2	20	S[1]
1	6	S[0]

top=-1

* اضافة عنصر الرقم (6) الى المكدس S؟

① top==Size-1
= 5-1=4
-1 ≠ 4 غير صحيح

② top=top+1
=-1+1=0

③ S[0]=6

* اضافة عنصر رقم (20) الى المكدس ؟

① top ≠ 4
0 ≠ 4 صحيح
top=Size-1
=5-1=4 ≠ 0 لا، ضفنا الجديد بدون تحريك القديم

② top=top+1
=0+1=1

③ S[1]=20

* قارن بين عملية سحب عنصر (pop) وعملية اضافة عنصر (push)

① التحقق من عدم فراغ المكدس من العناصر لتجنب حالة الفيض (underflow) عندما (top==-1)

② سحب العنصر من الموقع الذي يشير اليه المؤشر (top)
item=S[top]

③ تحديث قيمة المؤشر top=top-1

* Top==Size-1 المكدس ممتلئ تجنب عملية الاضافة => overflow

* top==-1 المكدس فارغ العناصر تجنب عملية الحذف => underflow

الموضوع: الملحق بالمكدس

التاريخ: سؤال الكويز

* ادخال عنصر الى المكدس ⟸
* اخراج عنصر ⟸

Q: Let (S) is stacking
(U) is un stacking
Inputs: 1 2 3 4 5
find the resuhts of: ① SSUSSUSUUU
② 32415

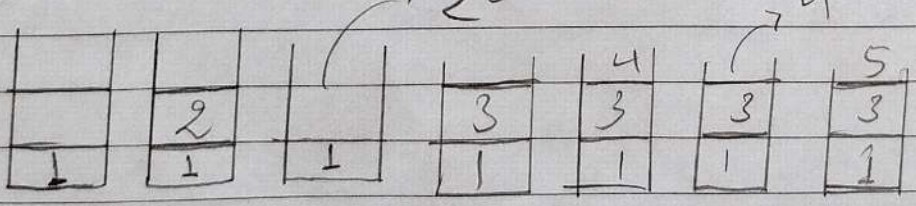
الحل ① SSUSSUSUUU

Input	1	2		3	4		5			
Operetions	S	S	U	S	S	U	S	U	U	U
Output			2			4		5	3	1

طريقة الحل: أول شي أسوي جدول راح اخلي بي العملية مال S و U بعدين اجي اشوف اول عملية مثل ادخال S راح ادخل اول عنصر من السؤال (الناحي "Input") بعد هنا خلصت، ملحة العنصر الاول بالعمارة" بس خلت الادخال «يعني ما اكدر ادخله مرة ثانية» بس اكدر اخرجه من تحتي عملية اخراج U هنا لازم اخرج اخر عنصر دخلته بالجدول وهكذا بقية الكل.

* اسوي اخراج لآخر عنصر دخل بحقل الـ Input

ثانية الحل:



الموضوع: ⊛ هاي الحالة جاية بالفايل %100

التاريخ: / /

② 3 2 4 1 5

الـ Input 1 2 3 4 5

Operations S S S U U S U U S U

Output 3 2 4 1 5

طريقة الحل: 3 2 4 1 5 → S S S U U S U U S U

1 2 3 3 2 4 6 1 5

لازم اضافتين حسب 1 2 3 4 5
حتى اكمل اخرج

⊛ سؤال فكري بالنهاية

يجي / يدخل الدكتور مخرج لا يمكن الحصول عليه باستخدام المكدس (مخرج خاطئ)

سؤال أصلي السؤال: عدد المخرجات الصحيحة والمخرجات الخاطئة باستخدام المكدس

التاريخ: / /

الموضوع: Lec 4

تطبيقات المكدس

• Stack Applications

Mathmadix Expressions

① A + B ⟸ Infix notation (بين)

- A: معامل / operand
- +: operators عملية حسابية
- B: operand معامل

- (بين) عبارة عن عملية حسابية تأتي بين معاملين.

② + A B ⟸ Prefix notation (قبل)

- operators: العملية الحسابية
- operands: المعاملات (بعد العملية)

- العملية الحسابية قبل المعاملات Pre
- عرف معنى prefix.

③ A B + ⟸ Postfix notation (suffix)

- operands
- operators

- كل عملية حسابية يحتاجها معاملين
- العملية الحسابية تأتي بعد المعاملات

* لأجل محاكاة التعابير الحسابية يقوم المعالج بتحويل التعابير الحسابية من صيغة Infix الى صيغة postfix لأجل ايجاد النتيجة النهائية وتعتبر هذه العملية أحد اهم تطبيقات المكدس. (يعني المعالج يستخدم المكدس حتى يحول التعبير الحسابي من Infix الى Postfix)

① Using Stack to Convert Infix to Postfix

أستخدام المكدس لتحويل التعبير الحسابي من Infix الى Postfix

- الصيغة الاساسية ل Postfix تتفوق كثيراً في أستخدام المكدس بالعمليات الرياضية لأنها بعد تحويلها تصبح جاهزة للحساب

ex: 3+(2×5)=13

(3+2)+5=10 → 32+

الموضوع: _____ التاريخ: / /

خوارزمية تحويل التعبير الحسابي من Infix الى Postfix باستخدام المكدس:

① تهيئة مكدس خالي من العناصر وسلسلة postfix تحتوي على النتيجة النهائية
② يتم قراءة عناصر التعبير الحسابي من اليسار الى اليمين رمزاً رمزاً.
③ اذا كان الرمز:
① معامل: يتقل الى المخرجات ((معامل يعني A أو B أو أرقام))
② علامة حسابية: افراغ جميع العمليات من المكدس الى سلسلة المخرجات والى اسبقيتها أعلى أو تساوي العملية الحديدة ثم ادخال العملية الحديدة الى المكدس
((يعني لو في عمليات موجودة بالمكدس اسبقيتها اعلى او تساوي العملية الحديدة اردخلها اطلعها الى سلسلة المخرجات))
③ قوس يسار «مفتوح»: تدخله الى المكدس.
④ قوس يمين «مغلق»: سحب جميع العمليات الموجودة في المكدس الى سلسلة المخرجات وحذف القوس اليسار حيث يتم اقراءة والعملة (عند قراءة اقفل القوس اليمين)
④ عند انتهاء قراءة التعبير الحسابي: افراغ جميع العمليات الى سلسلة المخرجات من المكدس.

الأسبقية
1 أعلى من الأس ← الأس ^
2 الضرب والقسمة ، AND ، الجمع والطرح ، OR
3

* الأسبقية بالمكدس صحيحة

الموضوع: _____ التاريخ: / /

* المكدس يستخدم فقط لأدخال العمليات الحسابية.
* المعاملات الى قائمة المخرجات مباشرة".

* المثال الاول صحة و بالمذكرة حالة "خرأ وخطة خرأ"
7 حالة المكدس (الخوارزمية تكون اقوى لتوضيح العمليات في المكدس والتي اسقطها على "وشاوي" عدا هذا الشكل

-	(	*	+

* حسب قانون المكدس سيصير أقوى عنصر قبل عنصر سيصير أقوى عملية الضرب اي هنا قبلها قوس لازم يخرج اول مرة القوس يعدين يخرج الضرب

* القوس المفتوح يخرج المفروض ما يكون اكو قوس مغلق
* القوس لا يقارن مع العملية الحسابية

* دائماً لازم نقارن صحة المكدس لأنه بس لحد صرير أقوى الباقي

Q: $A+B*(C^n D-E) \Rightarrow ABCD^nE-*+$

2^{1^3} 2^3

* مقارنة C^D

الخطوة	الرمز	الوصف	حالة المكدس	Postfix
1	A	معامل	+ #	A
2	+	عملية	+	A
3	B	معامل	+	AB
4	*	عملية	+ *	AB
5	(	قوس مفتوح	+ *(	AB
6	C	معامل	+ *(	ABC
7	^	عملية	+ *(^	ABC
8	D	معامل	+ *(^	ABCD
9	-	عملية	+ *(-	ABCD^
10	E	معامل	+ *(-	ABCD^E
11	)	قوس مغلق	+ *	ABCD^E-
12	انتهاء العملية			ABCD^E-*+

الموضوع: _____ التاريخ: / /

↑ الأعلى: الأس ^
③ NOT
② AND / % الضرب / القسمة
① OR / الجمع / الطرح

② حساب صيغة Postfix باستخدام المكدس:

الخوارزمية:-

① نستخدم مكدس لتخزين العاملات.
② نقرأ رموز التعبير الحسابي:
① إذا كان معامل: نضعه في المكدس.
② عملية حسابية: تنفذ العملية على أخر عنصرين في المكدس ثم النتيجة أو نعيدها الى المكدس
③ بعد انتهاء قراءة رموز التعبير الحسابي فإن النتيجة النهائية ستكون مخزونة في المكدس

$A_3 + B_4 \times (C_2 \hat{} D_3 - E_4)$

$3 + 4 \times (8 - 4)$

$3 + 16$

19

ABCD^E-*+
3423^4-*+

④ كل عملية أنفذها دائما على آخر عنصرين ونتيجتهم أضعها للمكدس كل مرة

الموضوع: _____ التاريخ: / /

الرمز	المكدس
3	3
4	34
2	342
3	3423 → $2^3 = 8$
^	348
4	3484
-	344
x	316
+	19

* هل محتويات الstack أكبر أو تساوي الرقم المدخل؟
نعم المحتويات تخرج للـ output
لأن المحتويات والرمز المدخل يبقيان في الـ stack

* لازم اسأل نفسي سؤال اشوف العملية اللي اريد اوجدها هل هي أعلى من القيمة حال السؤال
اذا اي هي اعلى نقول "هنا اخر العملية" بالـ stack عادي لأن هي اعلى من قيمة الـ stack القديمة.
لكن !! اذا كانت أقل أو تساوي صير اطلعها بالـ stack هنا اطلع اي بالـ stack الى اقرى انه اي تخير هذه العملية اعلى لا اكدر اطلعها بالـ stack

* لا يمكن وضع عامل ذو أولوية منخفضة فوق عامل الأولوية "العالية" نسحب الأعلى من المكدس ونحلل بالمخرجات ونضل نعمل هيك بالمثال

(* ^) → ^ *

* اذا كنت عملية حسابية سابقة اقاربها بالكبر اذا الكبر او تساوي اطرحها الى المخرجات

التاريخ: / / حساب

الموضوع:

Q: Compute the following Expression Using Stack $3*2+6/2-3^2$

* ملاحظة: اذا اعتبي بصيغة السؤال كلمة Convert الي معناها تحويل هنا الكل يكون هو اول ما يجيبه infix الى صيغة postfix اما اذا اعتبي كلمة Compute الي معناها حساب هنا اول شي اسوي اول ما احول من infix الى postfix بعدين اي اصير صيغة postfix

* Convert : تحويل $3+2$ تحويل $32+$ ("infix" "postfix")
* Compute : حساب (انتظري تابع)

Sol :-

الخطوة	الرموز	العملية	حالة الكدس	Postfix
1	3	معامل	#	3
2	*	عملية	*	3
3	2	معامل	*	32
4	+	عملية	+	32*
5	6	معامل	+	32*6
6	/	عملية	+/	32*6
7	2	معامل	+/	32*62
8	-	عملية	-	32*62/+
9	3	معامل	-	32*62/+3
10	^	عملية	-^	32*62/+3
11	2	معامل	-^	32*62/+32
12	انتهى		#	32*62/+32^-

الموضوع: _____ التاريخ: / /

3 2 * 6 2 / + 3 2 ^ -

الخطوة	الرمز	الوصف	حالة المكدس
1	3	ضع 3 في المكدس	3
2	2	ضع 2 في المكدس	3,2
3	*	العملية 3*2=6	6
4	6	ضع 6 في المكدس	6,6
5	2	ضع 2 في المكدس	6,6,2
6	/	العملية 6÷2=3	6,3
7	+	العملية 6+3=9	9
8	3	ضع 3 في المكدس	9,3
9	2	ضع 2 في المكدس	9,3,2
10	^	العملية $3^2=9$	9,9
11	-	العملية 9-9=0	0

Lec5 (Recursion)

* تعتبر ثالث تطبيق للمكدس

تطبيقات المكدس
1. تحويل
2. صيانة
3. الاستدعاء الذاتي

* شرط الـ infix يحوّل على القوائم على القوائم بالـ infix من تريد تغيير الأسبقية اذا قفل القوائم يعني الأسبقية عل حالها

الموضوع: _____ التاريخ: / /

Recursion:

هو قابلية البرنامج الفرعي لاستدعاء نفسه ولذلك سوف تعيد الدالة تكرار نفسها او تستدعي الدالة نفسها عدد إذ يتم إستدعاءها داخل الدالة يقيس لحد الدالة ولكن بقيم مختلفة إلى ان يتحقق شرط التوقف للدالة المستدعاة ويستخدم الأستدعاء الذاتي عبارة

- If - Then - Else

حلت المشكلة ← تحقق شرط التوقف ← القانون

If (stopping condition satisfied) Solve it
Else Reduce Problim using Recursion
أختبر بالكل اذا لم يتحقق الشرط

تبسيط المشكلة باستخدام الاستدعاء الذاتي

② / compare between loop and Recursion

قارن — المعاد

Loop	Recursion
① تستخدم for, Do, while	① يستخدم If-Then-Else
② يحتاج وقت وجهد اكبر من المبرمج (خطوات اكثر)	② يوفر الوقت والجهد للمبرمج (طول الخطوات أقل وأسرع)
③ وقت تنفيذ أقصر	③ يحتاج وقت تنفيذ أطول؟
④ مساحة خزنية أقل	④ يستهلك مساحة خزنية كبيرة؟
⑤ يحتاج الى قيمة نهائية للتوقف	⑤ يحتاج الى شرط توقف
⑥ أسرع	⑥ أبطئ للمعالج بسبب الأستدعاء المتكرر للبرنامج الفرعي

بسبب الأستدعاء المتكرر للبيانات

لان يستخدم المكدس

الموضوع: _____ التاريخ: / /

س: ماسبب استخدام الأستدعاء الذاتي بالرغم من استهلاكه للمساحة الكبيرة؟

الجواب: لانه يوفر الجهد والوقت.

س: لماذا استهلاك مساحة؟

الجواب: لاستخدامه المكدس.

* من أهم شروط تحقق عمل برنامج الأستدعاء الذاتي هو توفر شرط التوقف.
* اذا تحقق شرط التوقف تم حل المشكلة. (*) Recursive Stop طريقة مرادفة للعداد
س: لماذا يتم استخدام الأستدعاء الذاتي على الرغم انه هناك هدر بالمساحة

الجواب: لانه يوفر الوقت والجهد (لان ظروف قليلة بعض المشاكل يحتاج لها ظروف جد كبيرة فاستخدام الاستدعاء حتى يوفر لك الجهد وقت وخطوات اقل

Ex: Comput X! مضروب أي عدد → 4! = 4x3x2x1 = 24

باستخدام العداد For

```
int fact(int n)
{ f=1;
for (i=1; i>=n; i++)
f=f*i
Return (f)
}
cout << fact(4); => 4
```

باستخدام الاستدعاء

```
int fact2 (int n)
{if (n<=1) return 1;
else
return (n*fact2(n-1));
}
```

n*(n-1)*(n-2)*(n-3)
4 3 2 1

الموضوع: الطابور

Queue

التاريخ: الطابور باستخدام المصفوفة

هو هيكل بياني خطي من النوع الثابت يتم تمثيله في الذاكرة بصورة مؤقتة ويعمل وفق مبدأ من يدخل أولاً يخرج أولاً « First In first out »

("static queue")

لأجل بناء الطابور الثابت يتم استخدام المصفوفة احادية البعد لحجز الطابور « Size » يتم استخدام مؤشرين أحدهما للإضافة وهو R - « Rear » والآخر للحذف أو سحب عنصر وهو f - « front »

R = Size - 1
(rear)

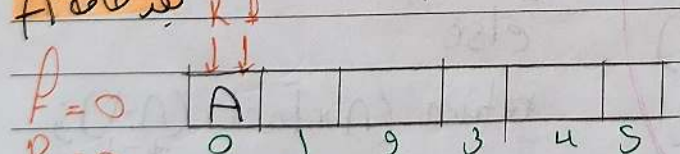
* متى يمتلئ الطابور؟
* الطابور الخالي من البيانات؟

front = -1

Q Size = 6

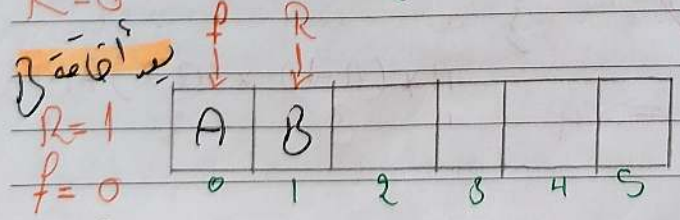


بعد إضافة A



R = R+1 , -1+1=0
f = f+1 , -1+1=0

بعد إضافة B



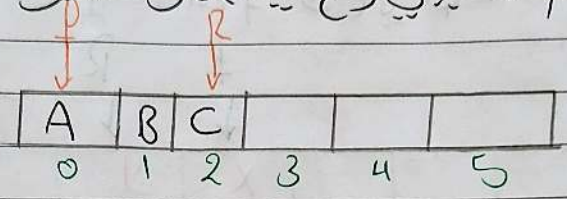
R = R+1 , 0+1 = 1
f = 0

("fifo", "LiLO")

الموضوع: _____ التاريخ: / /

* فقط عند أضافة أول عنصر إلى الطابور يجب تحديث قيمة f أيضاً، يعني راح يتحدث اول شرطين f و R

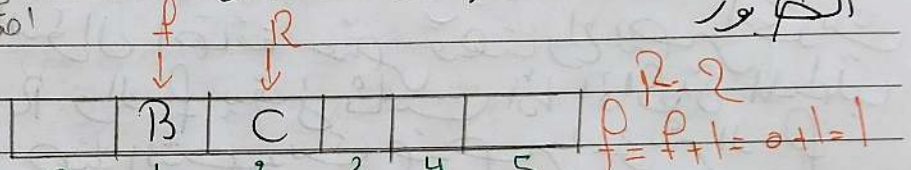
لإضافة C
R=2
f=0



R=R+1=1+1=2
f=0

* ما يحدلي متى بعد لازم أتأكد متى أسحب أشي على صح أسحب أول عنصر وصل الى الطابور

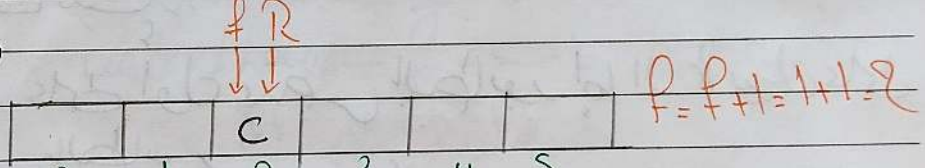
لسحب عنصر "A" — اللي اضفتها



R=2
f=1

R=2
f=f+1=0+1=1

لسحب عنصر "B"



R=2
f=2

f=f+1=1+1=2

* اذا "f=R" معناتي انتي وصلت لآخر عنصر بالطابور من اوصل لهاي الحالة اكدر اضيف لكن بالسحب ما اكدر فلازم اتأكد متى ارجع من البداية ضرع اسوي قيمة

f=-1
R=-1

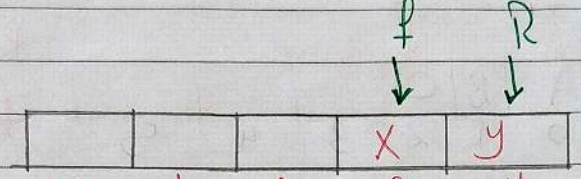
* عند حذف آخر عنصر من الطابور بحيث تصير مؤشرات الطابور بحيث f=-1 و R=-1 "اعرف آخر عنصر من f=R"

f=R=-1

الموضوع: / / التاريخ:

Underflow :- تقدر سحب عنصر من الطابور يعني انه front=-1 وهي حالة الشيء.

ex:- سؤال



الجواب :- R=4 f=3

* اذا طلب السؤال اضافة عنصر فهنا راح يصير التحديث يمشي الـ R والـ f يبقى ثابت أما اذا كان الطلب حذف عنصر هنا التحديث حيصير يمشي الـ f والـ R يبقى ثابت.

* الـ f يدل على أول عنصر بالطابور إما الـ R يدل على آخر عنصر بالطابور

* اذا طلب الكمبيوتر تنقل الطابور بالأعلى اضافة عنصر m الى الطابور الجواب حيكون تقدر عملية اضافة عنصر الى الطابور لان R=Size-1 لأن الطابور ممتلئ الناتج Overflow =4

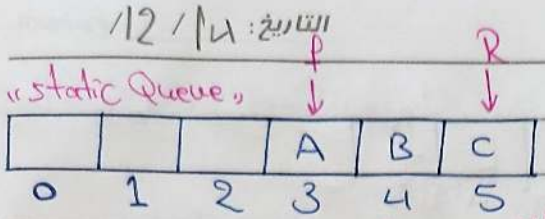
* تستخدم الطابور الدائري الي هو فائدته انقلال المساحة انقلال أمثل ويتم التخلص من صعوبة اضافة عنصر الى الطابور بالرغم من وجود اماكن خالية في مقدمة الطابور

* تقدر أكمل استخدم المساحة؟ استخدم طابور دائري لمثل

* أصعوبة الطابور الغايت هو أكو هدر بالذاكرة.

الموضوع: Circular Queue

التاريخ: 12/14



* بهذا المثال الطابور ثابت اذا طلب الدكتور اضيف عليها أكثر من اللي فيه لان بساحة الطابور ممتلئ حتى لو كان هنالك اماكن فارغة"

Q: size = 6

R = size - 1

= 6 - 1

5 = 5 ← ممتلئ

* حسب نظرة المعالج R = size - 1 فأن الطابور ممتلئ

بس نكدر نجيب قيمة أول عنصر الي هو A

* من أهم كون الطابور الأعتيادي الثابت هو عدم الأستغلال الأمثل للذاكرة مثل تعذر أضافة عناصر جديدة الى الطابور بالرغم من وجود مواقع خالية في المقدمة. لأجل حل هذه المشكلة يتم استخدام الطابور بصورة دائرية.

- ممتلئ عند R == size - 1 overflow — «مو أريد أضيف نقصص بالحارة»
- فارغ عند f == -1 underflow: — «مو أريد أفصص على الحارة اسحب»

«قوانين الطابور الأعتيادي»

* يتم استخدام الطابور دائرياً عندما

① عندما R = size - 1 يجعل مؤشر الـ R يساوي 0 R = 0 يأشر على موقع صفر

* اول شي بعملية الاضافة لازم أجيب ممتلئ أولاً.

* اذا كان f ≠ 0 أو اكبر من صفر معناها أكدر اضيف R على موقع صفر.

«قوانين الطابور الدائري»

- الطابور الدائري ممتلئ عند

① R == size - 1

شرط «و» ← && ← «و»

② f - R == 1

أو

OR

R == f - 1

f == R + 1

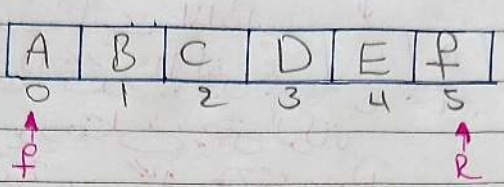
* واحدة من هذي الشروط تتحقق

* شرط السطرين الأخيرين يتحققون القفرتين بهاي الحالة لازم يتحققون

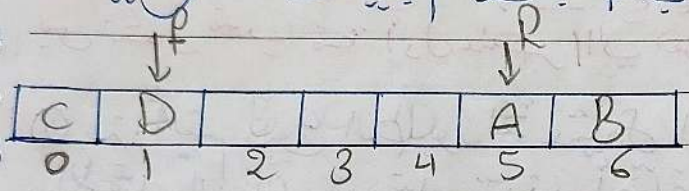
* اذا تحقق حالة وحدة من الي فوك معناها الطابور الدائري ممتلئ

الموضوع: _____ التاريخ: / /

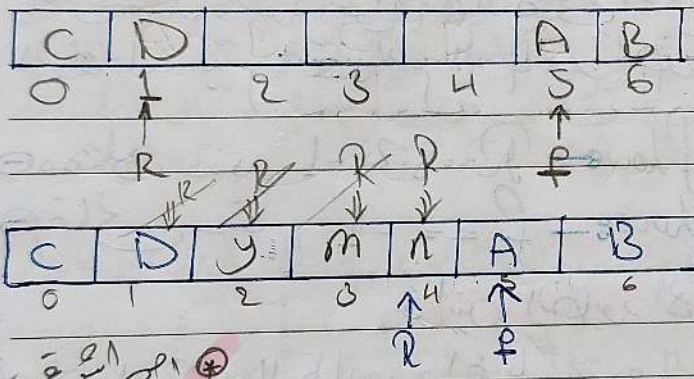
ex:-



Q: مثل الطابور الدائري سعته 7 باستخدام بيانات افتراضية اذا كان



R = 5
f = 1



R = 1
f = 5

R = size - 1 → 1 = 7 - 1
R = R + 1 = 1 + 1 = 2 → اضيف عنصر y (1 ≠ 6)
R = R + 1 = 2 + 1 = 3 → m
R = R + 1 = 3 + 1 = 4 → n

* اثبات الشرط
* متتابع f - R == 1
5 - 4 = 1 → 1 = 1 ✓

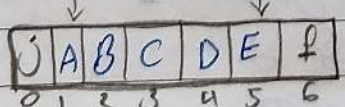
* f == R + 1
5 = 4 + 1
5 = 5 ✓

* R == f - 1
4 = 5 - 1 → 4 = 4 ✓

* مثالي الطابور الدائري

- الطابور الدائري فارغ عند f == -1

* يتم الحالة الخاصة في الطابور الاعتيادي او الثابت



R = size - 1
0 = 7 - 1 → 0 ≠ 6 → الحالة الاولى متحققة
f ≠ 0 → تحققت في حال امتلاء



تحقق على الحالة الثانية
R = f - 1 → 0 = 1 - 1
0 = 0 ← تحققت في حال فراغ

الموضوع: تكملة المحاضرة الماضية "7"

التاريخ: / /

للدور أن الى النهاية الامامية للطابور عندما يكون هنالك مواقع خالية ويجد هذا الطابور على عدة من الخواص:

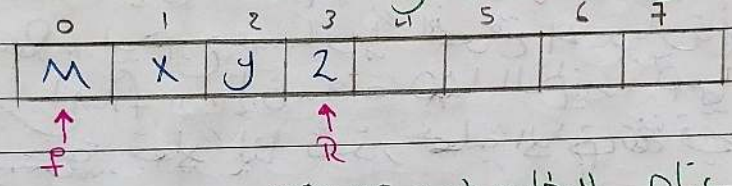
① ان المؤشر "Rear" يشير الى موقع العنصر الاخير في الطابور.

② عندما يصل المؤشر "Rear" الى الموقع الاخير في الطابور أن Rear == size-1 فأنه يدور الى البداية Rear==0 وكذلك بالنسبة الى المؤشر "front".

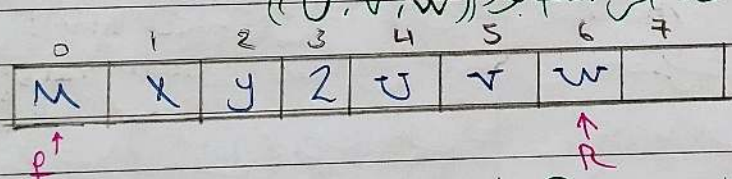
③ أن أكبر عدد من العناصر التي يتسعها هذا الطابور هو size-1

مثال(35)/ لدينا الطابور الأتي الذي يتكون من ثمانية مواقع خزنية:

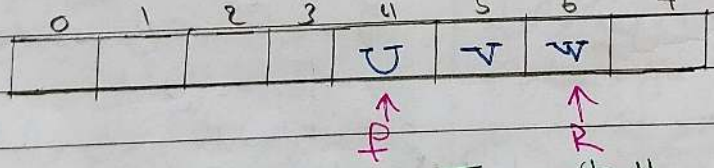
1- أخذ 4 عناصر للطابور ("M,X,y,Z")



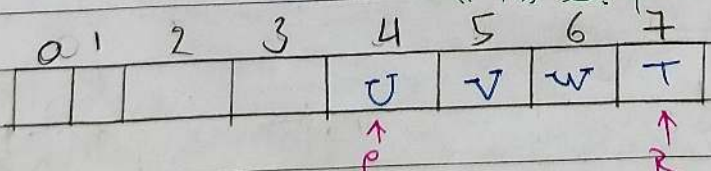
2- أخذ ثلاثة عناصر للطابور ("U,V,W")



3- أحذف العناصر "M,X,y,Z"



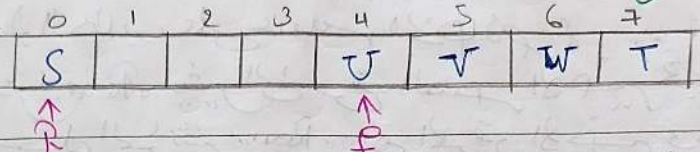
4- أخذ عنصر للطابور "T"



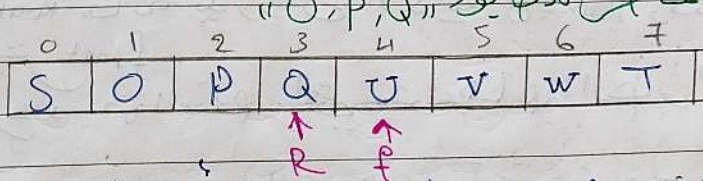
لاحظ هنا في الطابور البسيط يعتبر الطابور ممتلئ لأن Rear=size-1 مع أنه يوجد مواقع خالية في مقدمة الطابور. أما في الطابور الدائري فأنه تستطيع اضافة عنصر في هذه الحالة الاخيرة

الموضوع: _____ التاريخ: / /

5- أحذف العنصر الأول من الطابور "S"



6- أحذف ثلاث عناصر من الطابور "O, P, Q"



بعد إضافة ثلاثة عناصر للطابور نلاحظ أن هذا الطابور أصبح ممتلئ لأننا لو أردنا إضافة عنصر فيصبح قيمة المؤشر "Rear" يصبح مساوياً للعدد 4 ويكون مساوياً لقيمة المؤشر "front"، إن سعة هذا الطابور هي Size-1=7 وهو ما يشير أنه ممتلئ لذا نقدر عملية الإضافة في هذه الحالة

الموضوع: Storage allocation التاريخ: / /

«التخصيص الخزني»

هياكل البيانات
- بسيط
- مركب
  - غير خطي
  - خطي
    - ثابت «static»
      - متجانسة
        - مكدس
        - طابور
      - غير متجانسة
    - متحرك «Dynamic»
      - عناصر موصولة «Linked list»
        - طابور
        - مكدس

static ← مقارنة بسيط

Dynamic

storage allocation
- خطي
  - static
  - Dynamic

«static»

1. خزن تسلسلي «الأول يتبعه الثاني الثاني يتبعه الثالث وهكذا بقية العناصر»
2. متعاقب «العنصر الثاني بعد الأول»
3. سهولة الوصول «عشوائي» ← سرعة Speed
4. سهولة التمثيل للبيانات ← سرعة Memory
5. مناسب للمكدس والطابور
6. سرعة عمليات الأضافة والحذف

Speed
Memory

«عوامل تؤثر بالخزن الثابت» عناصر تحليلية

7. يتطلب التعريف المسبق لحجم الهيكل البياني ← عند امتلاء الهيكل البياني Overflow

«Dynamic»

1. ~~«مؤشر»~~ X
2. غير متعاقب
3. عدم توفر خاصية الوصول العشوائي ← بهدف تحسين إدارة حيز الذاكرة الرئيسية بتخصيص العنصر المطلوب الوصول إليه
4. يحتاج مساحة خزنية أكبر من التخصيص الخزني الثابت
5. سهولة اجراء عمليات الاضافة الحذف والدمج
6. لا يحتاج التعريف المسبق لحجم الهيكل البياني
7. مناسب للمكدس والطابور.

كيف قارن بين التخصيص الخزني الثابت «static» والتخصيص الخزني المتحرك «Dynamic»

الموضوع: _____ التاريخ: / /

* الفرق الجوهري بين التخصيص الحتمي الثابت والمتحرك هو: الثابت "static": مقاماتيا
المتحرك "Dynamic": هو غير مقاماتي
لكن التوعين يبقون يتبعون مبدأ التسلسل

* الفرق الجوهري الثاني:
الثابت "static": يتطلب تقريغ مسبق لحجم الهيكل البياني
المتحرك "Dynamic": لا يتطلب التقريغ المسبق لحجم الهيكل البياني

* بالـ Dynamic يمكن تغيير حجم الهيكل البياني أثناء التنفيذ أما بالـ static لا يمكن تغيير حجم الهيكل البياني أثناء التنفيذ
علل: سبب عدم إمكانية تغيير حجم الهيكل البياني أثناء التنفيذ؟
ج: لأنه يتطلب التقريغ المسبق قبل التنفيذ لحجم الهيكل البياني (قبل التنفيذ لازم أحدد حجم الهيكل البياني)
علل: يمكن تغيير حجم الهيكل البياني للمتحرك أثناء التنفيذ؟
ج: لأنه لا يحتاج إلى تقريغ مسبق لحجم البيانات

* حتى تحقق هيكل بياني متحرك تستخدم المؤشرات

تقريغ مسبق
static

A	B	X	Y	m	n	stack Array
100	101	102	103	104	105	

BA=100

	①		②			③
	A		B			X
	100	101	102			105
⑤			④	⑥		
113			116	117		

Data	Link		Link		Link		
A	102	→	B	105	→	X	
100	عنوان العنصر التالي		102		105		n NULL
							آخر عنصر

الموضوع: _____ التاريخ: / /



محتوياته موقع
حقل التوصيل ← حقل للبيانات
* محتوياته عنوان العنصر التالي

Pointer & هو متغير محتوياته عنوان ذاكرة يتم تعريفه حسب المؤشر نوع البيانات التي يشير اليها

ex:- int *A
int *A
int* A

متغير من نوع مؤشر
محتوياته عنوان

int A = 10
5

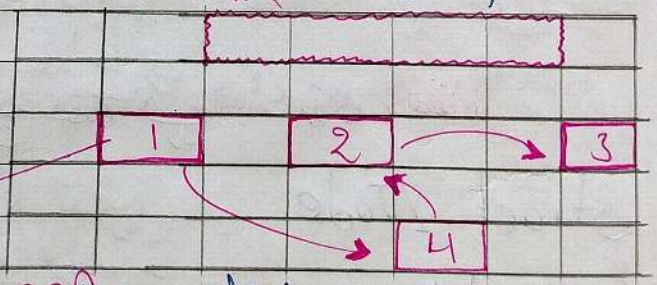
cout << &A
↓
(100)

* المؤشر محتوياته عنوان ذاكرة
* Refrense & عبارة عن عنوان ذاكرة لمتغير يربط عنوان الذاكرة الي محتوى هذا المتغير مع القيمة

Linked list

القوائم المربوطة ماعندي طريقة مثلثل

Static (sequential)



متغاث
غير متسلسل

Pointer هو متغير محتوياته عنوان ذاكرة & لمتغير ثاني نوع متغير المؤشر مشابه لنوع بيانات المتغير الثاني

int *p
p = &int

نوع البيانات int

الموضوع: _____ التاريخ: / /

القوائم الموصولة: هيكل بياني يتكون من مجموعة من العقد "Nodes"
كل عقدة تتكون من حقلين. الحقل الأول للبيانات والحقل الثاني هو رابط يمثل عنوان العنصر / العقدة الثانية في القائمة الموصولة.
تتميز بمجموعة من الخصائص أو الصفات:

1. الخلايا الخزنية متناثرة في الذاكرة (غير متعاقبة) حجم التعاقب
2. التخصيص ديناميكي (يحتاج مؤشرات)
3. الوصول تسلسلي (وقتي) مثل المصفوفة، وفي نفس الوقت خطي يطرأ في عملية الوصول للعنصر المطلوب
4. الحصر الخاص للعنصر الواحد لديه شيئاً (انتقالات أكثر للذاكرة) لأنه كل عقدة تتكون من حقلين على الأقل الأول لخزنة العنصر والثاني رابط للعقدة التي تليها
5. تتميز بمرونة عالية؟ لقدرتها على التقلص والتمدد أثناء وقت التنفيذ وحسب الحاجة إليها. (بسبب الإضافة والحذف)

* حتى أمثل هيكل بياني خطي ديناميكي لازم أول شي أنشئ عقدة وبعدين أكمل أنشئ عقدة ثانية

- الأسطر البرمجية لتعريف عقدة في الذاكرة:

* صيغة العقد هو متغير مؤشر تعريفه بالذاكرة يحتوي على أكثر من حقل من البيانات

```
struct Node
{ int data;
  Node *link;
};
struct Node *p, *q, *first, *start;
```

تعريف المؤشر عقدة

الموضوع: _____ التاريخ: / /

- أنشاء عقدة في الذاكرة:

```
=p=New node;
```

- لحذف عقدة:

```
=delete (P);
```

~~كيف أشرح هياكل البيانات الخطية الموصولة مع ذكر الخصائص~~

* لجواب هذا السؤال اول شي التعريف مال موضوعنا وثاني شي ذكر الخصائص واذكر الكود مال الاسم البرمجية لتعريف عقدة واذكر الاجراءات مال انشاء عقدة وحذف عقدة.

① البرمجية

```
struct student
{ Name
int dogree1, dogree2
float Avarage
}
```

* العمليات الموصولة من تعريف شوي قراءة تقوم التعليمة بعدهم

قراءة البيانات

```
cin>> student.Name
cin>> student.degree1
```

تعريف عقدة واحدة مع قراءة البيانات:

أول مرة انشئ عقدة بالذاكرة p→

data	Link
20	NULL

① `p==New node;`

② `cin>> p→data` هذا أدهن

`{p→data=20;`

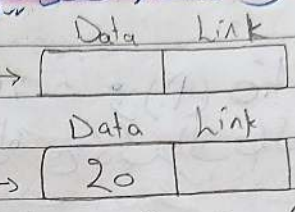
③ `p→link=Nuhhi;`

* العمليات الموصولة:
* استهلاك اكبر بالذاكرة
* حتى اعتل ديناميكي لازم انشئ عقدة
* الصفحة 96

الموضوع: _____ التاريخ: / /

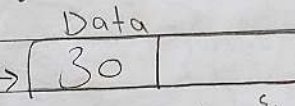
تعريف أو انشاء عقدتين في الذاكرة :

```
p = new node;
p -> data = 20;
```



* بعد أن أكون عقدة ثانية ما أكدد اقلي بحقل الـ Null link

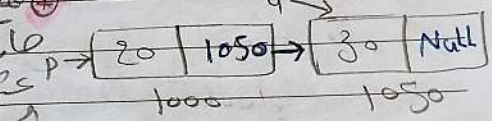
```
q = new node;
q -> data = 30;
```



* يبقى لازم أربط بينهم :

```
p -> link = q;
q -> link = Null;
```

* معنى الـ p عايز يحتوي على عنوان ذاكرة الـ q



* صار الربط بينهم

```
start = p;
```

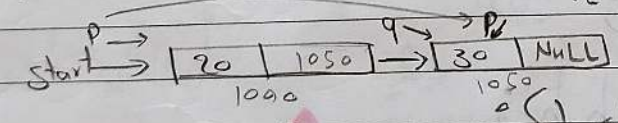
* خليت start في مطلع اول عنصر بالقائمة المطولة

* أعرف العنصر الأول بالقائمة المطولة من خلال p

```
p = = q;
```

* صار p يشير للعقدة الثانية

* لازم احفظ بمؤشر يشير الى بداية القائمة المطولة ومؤشر ثاني للبناء يتنقل بين العقد



* اذا طلب المكتور انشاء اكثر من 3 عقد لازم نعمل الحالة استخدم عداد * صفحة 99 مثال 5 لـ 5 انشاء عقد بشكل

* انواع القوائم المطولة بالمرة

* start هو نفسه first طريقة

* ما في فائدة من start + first الجواب / يشير الى اول عنصر في القائمة المطولة لبداية القائمة المطولة

الموضوع: _____ التاريخ: / /

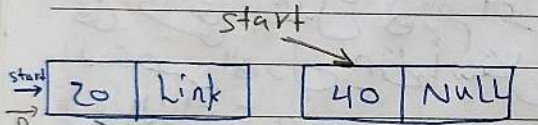
* القائمة الموصولة الدائرية تختلف أن العقدة الأخيرة تشير إلى العقدة الأولى «آخر عنصر أولى يشير إلى أول عنصر»
* بالسؤال السابق في القائمة الموصولة هي إجابة فاطلب الدكتور اسويها دائرية فقط أعتبر هذا الإجابة

q → link = start
p → link = start ← هذا الأهم

* القائمة الثنائية العنصر الأول يشير إلى العنصر الثاني العنصر البعدة يشير له بمعنى واحد يشير للثاني بالعكس

start → link = p
p → link = start ← واحد يشير للثاني

* مثال 100 عقدة «56»



* صف العقدة الأولى؟
* اصف العقدة الأخيرة في القائمة الموصولة؟

p = start
start = start → link; i=i+1 ← يعني دايكون أطفر للعقدة الثانية
delete (p);

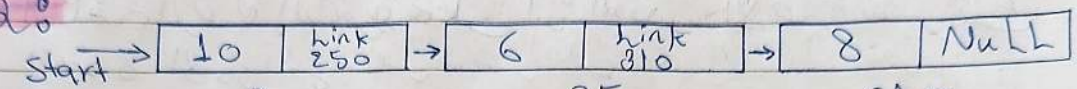
* p ⟸ للأشارة إلى أول عنصر
* q ⟸ للبناء «بنبني عقد»
* n ⟸ عدد العقد بالقائمة الموصولة"
* مثال التقرير العملي عقدة 100 بالنسبة لأخر موصولة"

cin >> q → data;

* نقرين عقدة بالذاكرة «صح»

الموضوع: _____ التاريخ: ٤/١/

Q:



① Add one node after postion 250, contains 66.
اضف عقدة بعد الموقع 250 محتواها 66

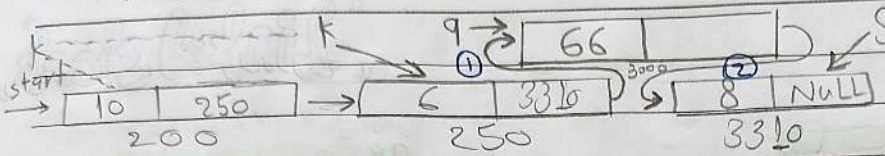
② Add one node at the beginning of list conigtains 70. اضف عقدة واحدة لبداية القائمة المرصولة تحتوي 70

"الهيكل البياني"

* كيف تعرف نهاية القائمة المرصولة؟ تحتوي على مؤشر Null
* كيف تعرف بداية القائمة المرصولة؟ تحتوي على مؤشر start يشير "الى اول عنصر في القائمة"
* أوصل لعناصر القائمة المرصولة عن طريق التسلسل لأنهم أعرف عنوان العنصر الاول عن أشكل الثاني وهكذا "راح تتخزن بمواقع متجاورة في الذاكرة"
* عن أوصل لباقي عناصر القائمة المرصولة لازم عشان وصلك لاول عنصر بالقائمة "المرصولة"
* المؤشر Null يشير إلى أخر عقدة في الهيكل البياني "نهاية القائمة المرصولة" لاقية حقل المؤشر Null
بينما المؤشر start يدل على أول عقدة في الهيكل البياني المرصول "بداية القائمة المرصولة"
* واحدة من اهم خصائص القوائم المرصولة أو المخزن الديناميكي هو سهولة اجراء عمليات التحديث الاضافة والحذف
* المخزن الخطي الديناميكي "معناه العنصر الاول يجي وراه الثاني وهكذا" الخطي الثابت "المصفوفة" خطي مرصول "قائم مرصول"

الموضوع: _____

التاريخ: / /



sol 8

إنشاء عقدة ← q = new node;

q → data = 66

② q → link = S

① k → link = q

sol 8

k = start

k = k → Link

① k → link = q

إنشاء عقدة * p = new node;

start → data = 20, 10, 5

sol 8 حل الطلب الثاني

حذف عقدة * delete(p)

تنزيح * p = p → Link

i = i + 1

① p = new node; إنشاء عقدة في الذاكرة

② p → data = 70 إضافة بيانات للعقدة 70

* q → Link = S = p ربط عقدة بعقدة ثانية

تشير إلى مكان ثاني

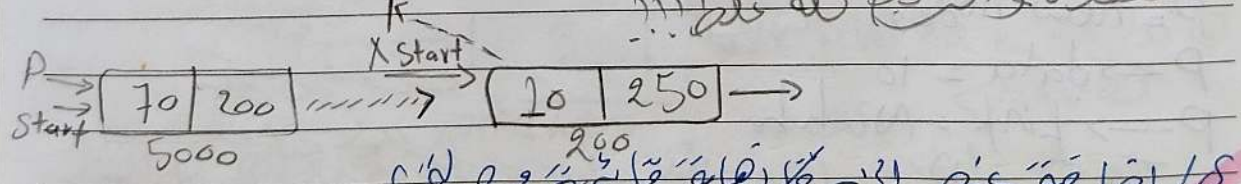
② p → Link = start ربط العقدة ببداية العائمة المرتبطة

start = p

* إذا كتبت p = start

يعني هنا عم بنقل الرم للعقدة الثانية وبيصير العقدة والربط كلة غلط!!!

ex:- X = 5



ع/ إضافة عنصر إلى نهاية قائمة موصولة

هنا اضف عقدة جديدة على 105 الى نهاية قائمة موصولة يشير المؤشر باسمة start + البداية

"يا بابا اكتمتان"

الموضوع: هكس موصول

التاريخ: / /

"Linked Stack"

هكس "مصفوفة"

① أول من يدخل أخر من يخرج "FILO" "first In last Out"

② الكتر المشابه "متجاور"

③ مصفوفة

④ الحجم ثابت ← يحتاج تعريف قبل تنفيذ البرنامج

ex: 10

⑤ فارغ عند: Top == -1

⑥ ممتلئ عند: Top = size - 1

⑦ نوع Top عند int

هكس "قوائم موصولة"

① أول من يدخل أخر من يخرج "نفس المبدأ"

② تخصيص حيزي متحرك "حسب الحاجة"

③ قوائم موصولة

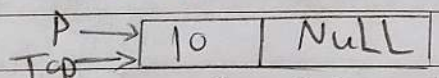
④ الحجم متغير ← لا يحتاج تعريف مسبق للحجم قبل التنفيذ

size ← متحرك

⑤ فارغ عند: Top == NULL

⑥ يمتلئ "يمكن اضافة عناصر لنهاية امتلاء الذاكرة"

⑦ نوع Top مؤشر Node *Top



① Top = Nuhh ← لانه اول شي بالبداية

p = new node;

p → data = 10

p → Link = Nuhhh

Top = p

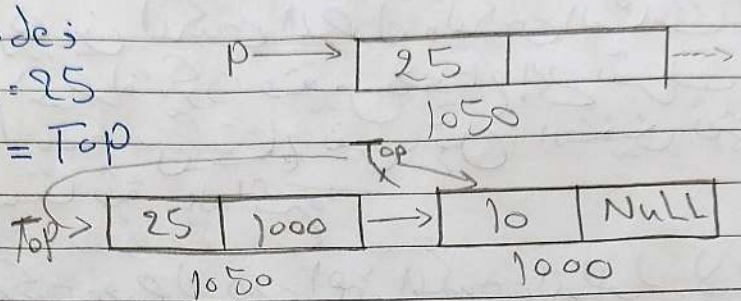
هكس يحتوي على عنصر واحد قيمته 10

"last In first out"

أخر من يدخل هو أول من يخرج

الموضوع: _____ التاريخ: / /

② p = new node;
p → data = 25
p → link = Top
Top = p



طيب:
Top = Top → link
item = Top → data
delete (P)

⟵ أخذت عقدة أم 25

Q: list 5 features related to Top in linked stack?
عدد 5 من صفات المؤشر Top في المكدس المربوط

① يشير الى أخر عنصر تم إضافة الى المكدس
② يشير الى أول عنصر يتم اخراجه من المكدس
③ يشير الى قمة المكدس "بداية القائمة المربوطة"
④ اذا كان Top = Null يعني المكدس فارغ من البيانات
⑤ ترجمة مؤشر Top الموجود هنا هو واحد موجود لو شايف هكذا فهو أحب
⑥ يخزن عنوان Address وليس بيانات

* أول عقدة تم اضافتها الى المكدس



Static stack: Top = size-1, Top == -1 "overflow" فارغ من
Dynamic stack: Top == Null "underflow" عملية الإدخال، لا يحتاج تحديد حجم قبل التعبئة overflow

الموضوع: _____ التاريخ: / /

* الفرق الجوهري بين المكدس الموصل والمكدس الثابت هو الحجم الحجم بالتعليمة الموصلة يكون متغير لا يحتاج إلى تعريف مسبق ليتم بالمرونة المكدس يحتاج تعريف قبل التنفيذ حتى المعالج يحجز أماكن مسبقاً بالذاكرة

* من أريد أبني مكدس موصل حتى أهيئ المكدس الموصل لازم أهيئ مؤشر Top

* المكدس فارغ حتى البيانات

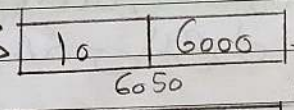
Top = Nulll

* انشئ موقع بالذاكرة محجوز على حقلين حقل بيانات وحقل للمؤشر

Top = New node;

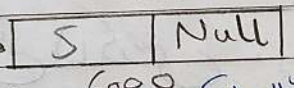
Top → data = 5;

p → link = Nulll;



* احتاج مؤشر أي يكون Top يشير الى نفس موقع ذاكرة الي يشير له p

Top = p



① أول عنصر تم ادخاله الى المكدس

② بداية المكدس جاهزة للبناء

p = New node;

p → data = 10

p → link = Top

* تربط العقدتين مع بعض

Top = p

* خلية Top تشير الى نفس موقع الي يشير له p

* حتى انقذ عملية حذف عنصر يشير له Top الي هو 10 لازم أحذف الـ p واعمل Top للعقدة الثانية

Top = Top → link

ضانقلت الـ Top

الموضوع: _____ التاريخ: / /

* باكتف لازم أجيب ضايع أولا

Top = Nuhh

If (Top == Nuhh)

cout << "stack underflow"

else item = P -> data ← اذا موضايي أول مرة لتنزل ال Top

Top = Top -> link ← i = i + 1

delete (P)

* عملية سحب موقتة، بالمعنى الحرفي "حذف عقدة من الكدس لحول"

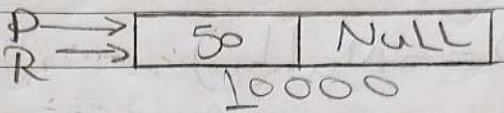
Dynamic linked Queue — مقارنة Compare — الطابور المتحول

Dynamic Queue	static Queue
① fifo يدخل اولا يخرج اولا	① fifo يدخل اولا يخرج اولا
② storage allocation Dynamic تخصيص خزني متحرك	② Static storage allocation تخصيص خزني ثابت
③ الحجم متغير	③ الحجم ثابت
④ لا يمتلئ	④ ممتلئ عندما R.size - 1
⑤ فاضي عندما f == Nuhh	⑤ فاضي عندما f == -1
⑥ f = R = Pointer مؤشر	⑥ f = R = integer عدد صحيح

الموضوع: "تهيئة طابور فاضي" التاريخ: / /

R=f=Nullptr;

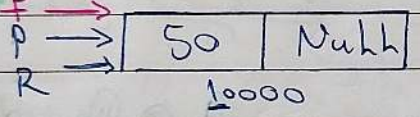
p=newnode;
p->data=50
p->link=Null



R=p
f=p
f=R=p
أضافة فيتي لازم مؤشر مباشر على Null وهو R لأن أضافة

* قبل كلشي وأول خطوة لازم تهيئة الحاوية المدخول
R=f=Nullptr;

Q: R=f=Nullptr;



بين مع ذكر السبب:-
① fulll ? ممتلئ
② Empty ? فارغ
③ Normal ? أعتيادي

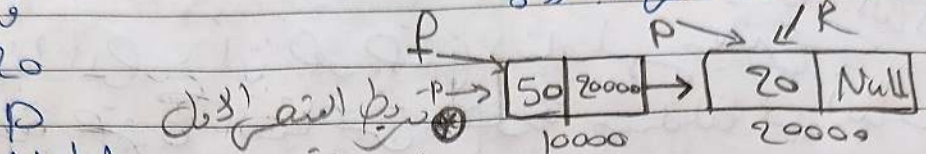
Soln:
① No → Dynamic ← لأن متحرك لا يمتلئ
② Yes → f=Nullptr ← حسب التعريف اللي بالاعلى والمعادلة
No → f≠Nullptr
③ No → f≠R
yes → f=R=P ← تكون جديدة وتكون ضيفي بيها أضافة مؤشر f

* الاضافة هذا الموضع تصير هيئة الحيث لأن R صوابي لازم يتحدث إذا عملتو الحيث شرح هيئة السيركلار من داخل أولا" كسر أولا"

الموضوع: _____ التاريخ: / /

أضافة عنصر بداية

```
p = new node;
p -> data = 20
R -> link = p
p -> link = Nuhh
R = p
```

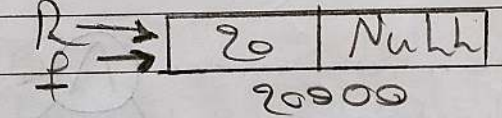


تربط العنصر الاول بالثاني

* لأن الـ R للإضافة قد مات والـ f بقيتا بالبداية حتى تستخدمة للسحب
* السحب يكون من الجهة الأمامية front والإضافة من الجهة الخلفية "Rear"

سحب عقدة من الطابور الطويل

```
p = f
f = R
f = f -> link
item = p -> data
delete (P)
```



* R -> link = f/p

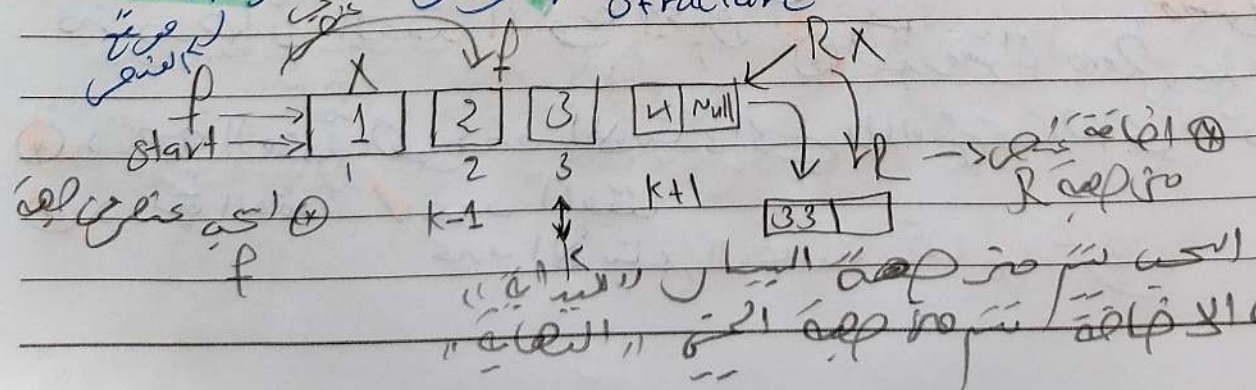
من خلال هذا الايعاز صيرت طابور دائري

وقت الوصول لعناصر ثابتة // Static structure

O(1)

وقت الوصول لعناصر // Dynamic structure

O(k)



الموضوع: الشجرة Tree التاريخ: / /

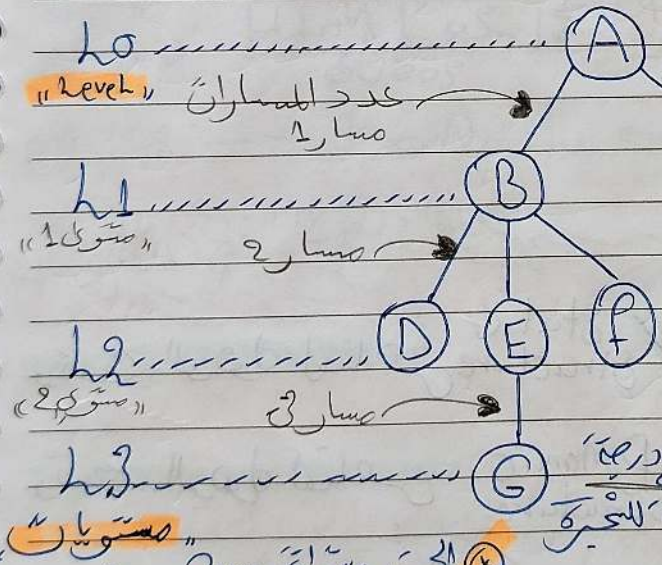
* أهم ميزة بهذا الموضوع أنه هو خزن ديناميكي مجتمع في الذاكرة لا يتبع طاهية خزن العناصر بصورة متسلسلة في الذاكرة
« خزن ديناميكي مجتمع »

هي هيكل بياني غير خطي لا يعتمد مبدأ التعاقب أو التسلسل في خزن البيانات في الذاكرة « خزن البيانات مجتمع في الذاكرة »

Root : هي العقدة الرئيسية في الشجرة
الجذر

Leaf : هي العقدة التي لا تمتلك أي تفرع « عقدة نهائية »
ورقة

ex =



A : هي
B,C : عقد أو أبناء لـ A
C,D : أوراق « ليس لديها تفرعات »
f و G

* مستوى العقدة « h0 h1 h2 h3 h4 »

* درجة الشجرة هو أعلى درجة من درجات العقد المكونة للشجرة « ك »

* الجذر مستواه Zero « h0 »

* درجة العقدة هو عدد الأبناء التي طالعين منها في العقدة

* درجة العقدة / عدد العقد
عدد الأبناء « الطالعة منها مباشرة »

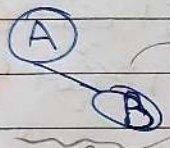
الموضوع: _____ التاريخ: / /

* **أرتفاع الشجرة**: هو أطول عدد من المسارات من المستوى الأخير «h» بمعنى هو درجة المستوى الأخير
فائدته: من خلاله أكتر أعرف أكبر عدد من العقد أقدر أخزنها بالشجرة «من خلال الارتفاع اعرف»

الشجرة الثنائية Binary Tree

هي شجرة كل عقدة فيها لا تحتوي على أكثر من تفريع أي درجة أي عقده فيها لا تزيد على اثنين»

* درجة الشجرة الثنائية هي «اثنين»
* هذا الرسم يمثل شجرة ثنائية هي صح ممكن تحتوي على أكثر من عقدتين بس ما لو شكلة اذا احتوت على عقدة واحدة فقط «ممكن اقل لكن ما يصير أكثر من 2»

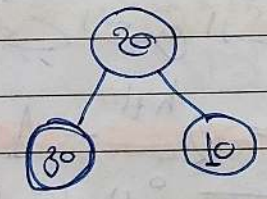


الشجرة الثنائية المتزنة Baianced Binary Tree

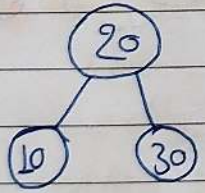
هي شجرة الثنائية اللي أي عقده فيها يكون لها أبناء اثنان «متوازنة»

«شجرة بحث ثنائية»

* الأبن اليسار تكون قيمته أقل من الأب والابن اليمين تكون قيمته أعلى من الأب
* أو الأب قيمته أعلى من الابن اليسار وأقل من الابن اليمين «الأب أقل من اليمين وأعلى من اليسار»
«الابن اليسار أقل من الأب الابن اليمين أعلى من الأب»



«شجرة ثنائية أعتيادية»



«شجرة بحث ثنائية»

التاريخ: / /

الموضوع:

ك: أذكر خصائص الجذر؟

ك: أذكر خصائص الأب في شجرة البحث الثنائية؟

حاصل صفات الأب في شجرة البحث الثنائية

① قيمة أعلى من الابن اليسار
② قيمة أقل من الابن اليمين

③ صفات أو صيغة الابن هي على السياق
④ قيمة أقل من الأب
⑤ قيمة أقل من الأخ

⑥ أكبر عدد من العقد في المستوى "2^L" أس $2L$ / "2^L"
هكذا إذا أريد العلم المستوى السوت 2^L
عكس يطلب 16 مثلاً أكبر رقم 16 عقدة بالمستوى الواحد
بمعنى
إلى صفر الجذر $2^0 = 1$
المستوى
Zero

⑦ أكبر عدد عقد في الشجرة الثنائية

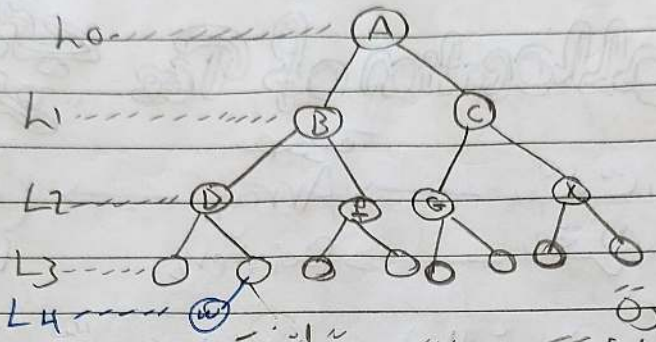
$$MaxNo = 2^{h+1} - 1$$

h: ارتفاع الشجرة

ex:-
$h = 3$
$MaxNo = 2^{3+1} - 1 = 15$ ← بمعنى أكبر أقل 15 عقدة كعدد أقصى في الشجرة

⑧ الشجرة الممتلئة هي كل المستويات ممتلئة كعدد واحد

الموضوع: _____ التاريخ: / /



هذا الشكل عبارة
عن شجرة ثنائية ممتلئة ومتوازنة

- تحتوي على 15 عقدة
- أرتفاعها يساوي 3
- درجة الشجرة تساوي 2
- درجة أعلى عقدة 2
- أكبر عدد من العقد على مستواها في هذه الشجرة هو 2^{h+1}
- العلاقة $Max_{no} = 2^{h+1} - 1$

حيث شجرة ثنائية ارتفاعها h

أي شجرة السؤال يعطيني كم عقدة أكبر أمثل بها بحيث أمثلها ارتفاع الشجرة هو أطول مسار فيها بالرسم القوي، اعرف مسار هو كم عقدة فيه h=3

$Max_{no} = 2^{h+1} - 1$

$= 2^{3+1} - 1 = 2^4 - 1 = 16 - 1 = 15$

أكبر عدد عقد

أكبر أمثلها أو
أرسمها بالشجرة لا أكثر ولا أقل

- مثال رسمت عقدة 63 صغير الارتفاع يساوي 4

$Max_{no} = 2^{h+1} - 1 = 2^{5} - 1 = 31$

ي: شجرة ارتفاعها 4 ما هو أكبر عدد من العقد يمكن تمثيلها في أي / أي عقدة

$Max_{no} = 2^{h+1} - 1 \Rightarrow 2^{4+1} - 1 = 2^5 - 1 = 32 - 1 = 31$

أكبر عدد من العقد يمكن تمثيلها في الشجرة

التاريخ: ٢٠٢٦/٣/٢٩

الموضوع:

"TREE"

Storage allocation of Tree تخصيص التخزين للأشجار "الشجرة"

1. static "ثابت" ← Array ← $(2^{h+1}-1)$ مصفوفة
2. Dynamic "متحرك" ← Pointers مصفوفة مربوطة

* الأشجار لا تتبع خاصية التسلسل ولا التعامل وأنها تكون مبعثرة في الذاكرة

1. static ← Array ← $(2^{h+1}-1)$ مصفوفة

$(2^{h+1}-1)$ ← تحجز مواقع بالذاكرة

1. الجذر في الموقع الأول للمصفوفة
2. $2*i+1$ الابن اليسار "الأصغر"
3. $2*i+2$ الابن اليمين "الأكبر"

* صيغ i هو عنوان الأب

تمثل شجرة تتبع الخوارزمية باستخدام هذه الطريقة الأشجار الثنائية

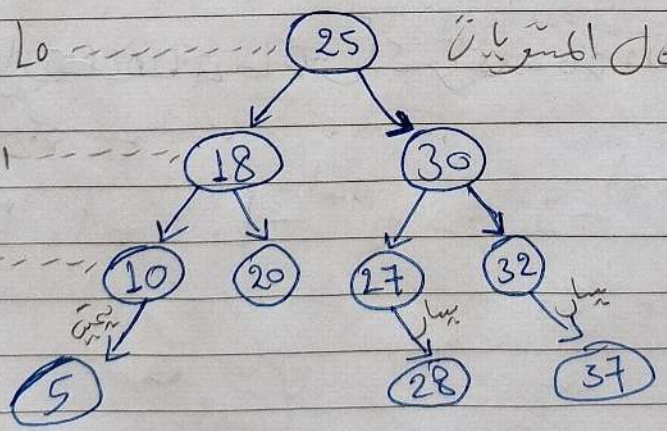
بشجرة البحث الثنائية

بشجرة البحث الثنائية

* هذه الخوارزمية تطبقها على شجرة ثنائية أو شجرة بحث ثنائية

S: عدد مواقع العقد في شجرة البحث الثنائية ؟

ex:-



* الـ h طبقة من خلال المستويات "h"

* أو الذي من عنده

i/2+1

i/2+2

الموضوع: _____ التاريخ: / /

8 6 8 -

* تمثيل الشجرة باستخدام المصفوفة:

Array Size = $2^{h+1} - 1 = 2^{3+1} - 1 = 16 - 1 = 15$

* يلزم أصغر مصفوفة بها 15 موقع بالذاكرة حتى أكد أمثل الشجرة بالمصفوفة.
* تأخذ فقط البيانات دون الشجرة تمثيلها بالمصفوفة
* تتسع ذاكرة مكونة من 15 موقع

25	18	30	10	20	27	32	5					28		37
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

مواقع الخلايا

* الابن اليسار 18 الأب طالة 25 موقعة 1 ← $2*1 = 2$
* شجرة البحث الثنائية تنقسم نصفين بالكامل ويبين العقد كل الجزء الي اليسار هم أقل من الجذر وأقل من الجهة اليمين

مميزات شجرة البحث الثنائية

+ علاقة الأبناء مع الآباء «علاقة» جاهزة»
* الجزء اليسار الشجرة بالكامل هو أصغر من الجزء اليمين وأصغر من الجذر «علاقة جاهزة»
* «الشجرة الترتيبية على الجهة اليسار للجذر أصغر من الجذر وأصغر الشجرة الترتيبية الموجودة على يمين الجذر»
* أي موقع الذاكرة أقسمه على 2 أو أقل واحد دخل موقع الأب والابن

الابن مال 18 موجود ← 18 = 2/2 = 1 موقع الأب

الترتيب الابن اليسار ← ابن اليمين الأول

موقع الابن الأب 30 = 3/2 = 1.5 = 1 عنوان الأب ناقة الرد الصحيح

27-6 ÷ 2 = 3

* اليمين قرني موقعه

10 = 4/2 = 2

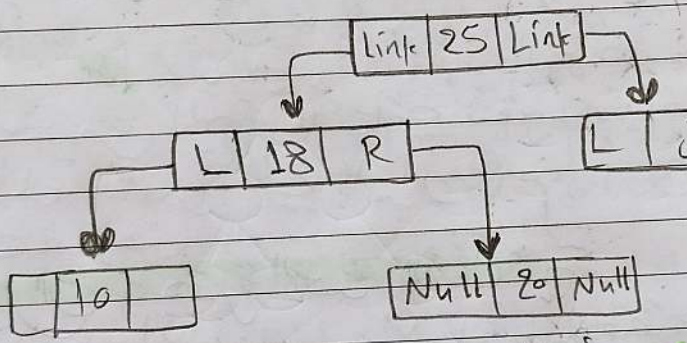
20 = 5/2 = 2.5 = 2

الموضوع: _____

التاريخ: / /

ك: أستخرج الشجرة الثنائية من الهيئة الصفية

- الوكيل يعطي رسم المرفقة ولازم أعمل عكس التجزء
- أول شي أعلى الجذر
- قاعدة الي عند الجذر هو أول 3 حقول اللي تمثل الجذر والابن اليسار والابن اليمين
- ثم أبتاد اليسار واليمين
- بالمثل لـ Dynamic
- بما أن هي شجرة ثنائية يعني كل عقدة عدها أتصال على الأكثر تفرع أو تفريعين لذلك كل عقدة لازم أكثر شي يبها رابطين



- العقدة الي ما عدها أبناء حط بدالها
- ما أضاع أمثل موقع ذاكرة فارغة راح أستغل فقط العقد الموجودة أمثلها فقط الحقيقي أعلم عقد البداية
- درجة الشجرة هي أكبر درجة للعقد «درجة الشجرة»
- شجرة اعتيادية درجتها 4 ثلوث أمثلها باستخدام الرمز المرسل
- h يعني الثابت

التاريخ: 2026/4/5

الموضوع: _____

* الشجرة الثنائية أكبر أولها إلى مجموعة أعادية
طرق التمثيل: ① استخدام المصفوفة
② باستخدام المؤشرات

* أكبر أعدد بموقع الأباء من خلال المصفوفة "حسابياً"
الموقع مال الإبن مقسمة على 2 نعرف الأب مالة

* بالمصفوفة عن طريق الأباء نكدر
نحدد الأباء

0	1	2	3	4	5
A	B	X	J		
1	2	3	4	5	6

* الأباء نحددهم عن طريق موقع الأباء
"موقع الأب * 2 إذا كان الابن اليسار"
"الابن اليمين i+1 * 2"

الي هو موقع A

B → 2/2 = 1 → موقع الأب

* عن طريق المصفوفة نكدر عدد بين كم فاصلة أبن يمين
نحدد الأباء

* إذا طلع الناتج عدد الابن صحيح فاصلة أبن يسار إذا طلع

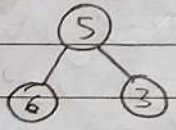
X → 3/2 = 1.5 → ① أبن يمين

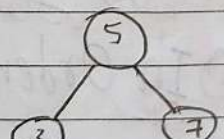
* إما عن طريق الشجرة نكدر
نحدد الأباء والأباء

address = i * 2

R = i * 2 + 1

* التمثيل المتصل "الموقع يبلش من واحد"
* التمثيل الفيزيائي "الموقع يبلش من صفر"
"تمثيل متصل"





* الشجرة شجرة ثنائية بس الي فوه شجرة ثنائية

زوجي ← أبن يسار
عدد صحيح "موقع الأب"

فردي ← أبن يمين
كسر

التاريخ: / /

الموضوع:

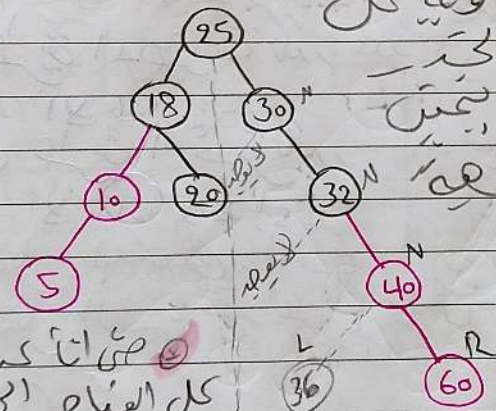
بناء الشجرة الثنائية

* لبناء شجرة ثنائية أريد أن أحولها الى شجرة بحث ثنائية
فما هي طريقة بناء شجرة بحث ثنائية

1. نضع العنصر الأول على أساس أنه الجذر "root"
2. كل العناصر الأخرى إذا كانت أكبر من العقدة الأولى "الجذر" تأخذ جانب اليمين
3. إذا كانت العقدة أصغر من العقدة الأولى "الجذر" تأخذ جانب اليسار

ex:- 25, 30, 32, 18, 20, 40, 10, 60, 5

* أقارن العدد الي اريد اطلي وية كل العقد ابتدي من عقدة الجذر الجذر هو اللي يحدد لي جهة يمين أو يسار باقي العقد الي الجهة هم لازم اقارن



* حتى انا كرمال على كل العناصر الي على جهة اليسار اعتبرها الجذر وحتى جهة اليمين

Tree traversing

أشهر طرق الشجرة

* تمثيل بيانات الشجرة يعتمد على موقع الجذر
* اسم طريقة الاجتياز

1. In Order method (LNR) (الأب اليمين الجذر الأب اليسار)
2. Pre Order method (NLR)
3. Post Order method (LRN)

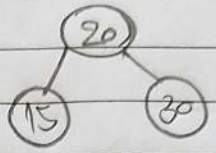
التاريخ: / /

الموضوع: _____

① In order method (LNR)
15 20 30

② Pre Order method (NLR)
20 15 30

③ Post order method (LRN)
15 30 20



* دائماً بيبقى الـ R أخير شي بتشوفه
* كل شجرة فرعية أعمل عليها الطريقة حال NLR او LNR

* استراحة مثال الشجرة السابقة
* اول ما اجي لـ 25 بعتبر هذا جذر أصغر الطريقة طلعت بالاستراحة هي اذا مشي لازم المشي في جهة اليسار بعد الـ 25 أكو العقدة 18 على جهة اليمين هنا الـ 18 أيضا جذر صح هو سيار بس هو جذر اليسار إلي تمثل ورقة عشان أكدر أفرقها
* لازم أبحث عن العقدة اليسار إلي تمثل ورقة عشان أكدر أفرقها

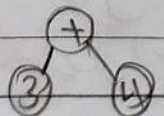
3+4
In order
LNR: 5 10 18 20 25 30 32 40 60
30 32 36 40 60

هذا الجزء الجذر الجزء الأيمن من الجذر تصاعدي
N تنازلي R

Pre order
NLR: 25 18 10 5 20 30 32 40 36 60
+34

Post order
LRN: 5 10 20 18 36 60 40 32 30 25
34+

* لازم كل مستوى تشتل بي بداية عشان تكون

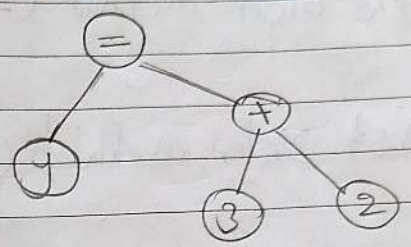


الموضوع: _____ التاريخ: 2026/4/8

استخدام شجرة البحث الثنائية في تمثيل التعابير الحسابية

Ex:- $y = 3 + 2$

L N R



* كل العمليات تمثلها الـ = تعتبر جذور لأشجار فرعية
* الـ = هي الجذر الرئيسي

* العملية هي الجذور

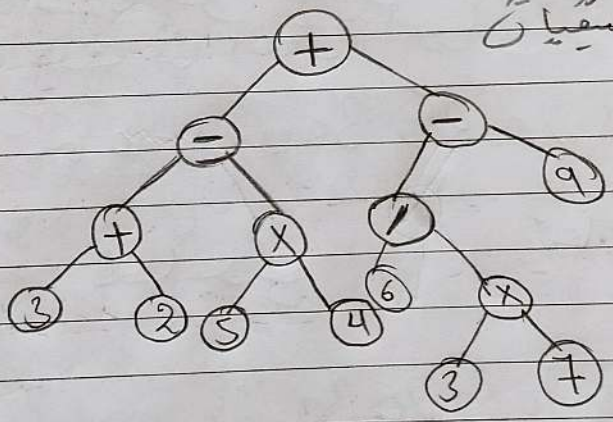
Q:- $3 + 2 - 5 \times 4 + 6 / 3 \times 7 - 9$

أولوية 2 — أولوية 1 — شجرة فرعية

* الجذر الرئيسي هو + الي بالنص

مطلوب تحويل هذا التعبير الى شجرة ثنائية

* لازم الاكس الأسفيانة



LNR: $3 + 2 - 5 \times 4 + 6 / 3 \times 7 - 9$

1. ضع عقدة بدون ابناء (ورقة) تحتوي صيغة
2. ضع عقدة لها إبن واحد العقدة تأخذ مكان الأب الذي يحتوي
3. اذا عقدة لها ابناء (2) تضع بها أكبر عقدة بالجهة اليسار لكل كل العقدة الطلوب وضعها

* هل اشجار التعابير الحسابية ممثلة ام متوازنة؟

لازم موازنة تقريبية

الموضوع: البحث Searching التاريخ: 12 / 4 / 2026

هو عملية البحث عن عنصر محدد ضمن مجموعة من البيانات.
نتيجة البحث إما وجود العنصر وموقعه أو رسالة بعدم وجوده. (1,0)

⊛ راح ندرس خوارزميتين لكن موجود
البحث نوعين الخطي والثنائي ندرس كمان هيك شو أفضل حالة وأسوء حالة نقدرها هاي الخوارزمية

① Linear Search البحث الخطي

هو أبسط انواع البحث ويعتمد مبدأ التسلسل في البحث عن العنصر المطلوب

⊛ راح نستخدم البحث الخطي لكن موجود خوارزميات أحسن منه
(يبدأ من العنصر الأول ثم الثاني ثم الثالث وهكذا)
مع مجموعة أعداد (حتى البرنامج)

25	30	10	5	60	70	6	3
0	1	2	3	4	5	6	7

لأنه مجموعة أعداد فبلش من موقع "0"

⑤ أريد البحث عن هذا العنصر أبلش من العنصر الأول إلى الموقع أقرب اذا هو أو لا هنا صار مقارنة طبعاً العنصر 5 منطقياً موجود بالموقع 3 هنا لازم الخوارزمية تقيدلي أن هو موجود وتقيدلي الموقع حالته واذا هو موجود تعطي رسالة غير موجود وتبلش "0" من هنا على جميع العناصر واذا ما موجود

ك: متى تتوقف الخوارزمية "برجع لفوق"
تتوقف خوارزمية البحث عندما تنتهي مقارنة العنصر المطلوب البحث عنه مع جميع عناصر المجموعة

الموضوع: _____ التاريخ: / /

خوارزمية البحث الخطي:-

① نبدأ بمقارنة العنصر المطلوب مع عناصر القائمة ابتداءً من العنصر الأول في القائمة.

② تستمر الخوارزمية بعد المرور على جميع عناصر القائمة أو عند العثور على العنصر المطلوب.

③ الـ complexity حال البحث بكافة حالات الخوارزمية:
- حسب حالات الخوارزمية أفضل نتيجة نرجعها حالة الخوارزمية وتكون كذلك هذا هو عندما يكون العنصر المطلوب هو موجود أول عنصر بالقائمة إما بالنسبة لآخر الحالات هذا إن العنصر المطلوب يكون آخر عنصر

البحث الناجح

Successful Search

Best result (أفضل نتيجة): O(1)

Worst result (أسوأ نتيجة): O(N)

UnSuccessful Search

④ متى يكون العنصر أعلاه غير موجود "المثل"

Best result: ⟵ العنصر موجود

Worst result: O(N) ⟵ أبحث عن كل العناصر وما لقيته يرجع نتيجة خطأ

الموضوع: _____ التاريخ: / /

البحث الثنائي ② Binary Search

يعتمد على مبدأ تقسيم البيانات الى مجموعات من أجل تسريع عملية البحث لكن بشرط أن تكون هذه البيانات مرتبة على سبيل المثال تصاعدياً. «يقلل من خطوات البحث»

* بالبحث الثنائي الفكرة انه أقسم القائمة إلى قسمين وأجي أقارن العدد المطلوب مع المنتصف اذا كان هذا العدد المطلوب أكبر من المنتصف يمشي على اليمين واذا أصغر من المنتصف يمشي على اليسار «يعني راح أهمل نصف القائمة من أول مقارنة» وانتقل للنصف الثاني مرة ثانية ايجاد القسمة بالنص وهكذا الى ان أوصل للعدد المطلوب

②			③			①			
13	20	30	50	60	80	100	120	140	200
X	X	X	X	X	X	X	X	X	X

(20) (35) (90) أصغر

* هذا النوع افضل من اي نوع من انواع عندي قوائم كبيرة

هذا

مثل يفضل استخدام خوارزمية البحث الخطي أو خوارزمية البحث الثنائي؟

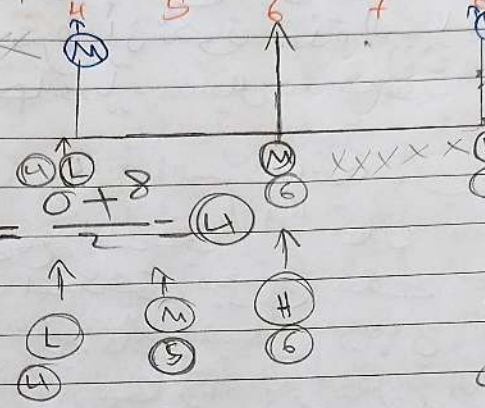
الخطي يفضل استخدامها بالقوائم الصغيرة اما الثنائية بالقوائم الكبيرة وما تحتاج اذا كانت مرتبة بالذاكرة

الموضوع: _____ التاريخ: ١٢ / ٩ /

ex: goal = 44

5	12	17	23	38	44	77	84	90
0	1	2	3	4	5	6	7	8

L: 0 الأصغر
H: 8 الأكبر
M: (L+h)/2 = (0+8)/2 = 4 الوسط



Best result: O(1)
Worst result: $\log_2 N = X$ $\log_2 8 = 3$

$\log_2 N = X$
$\log 8 = 2^3$

$2^N = X$ ← لوغاريتم

ملاحظة: ما نقدر نعرفها 1000 أصبح تعقيد الزمن في حال استخدام طريقة البحث الثنائية

Best: O(1)
worst:

ملاحظة: حال الأمثلة السابقة إذا انتقلت لمجرد البحث عطى مثلاً N عوض L وال H يبقى أما إذا جاء مكان N عطى H وال L يبقى مكانه

ملاحظة: أس لـ عشان يطلعلنا 1000

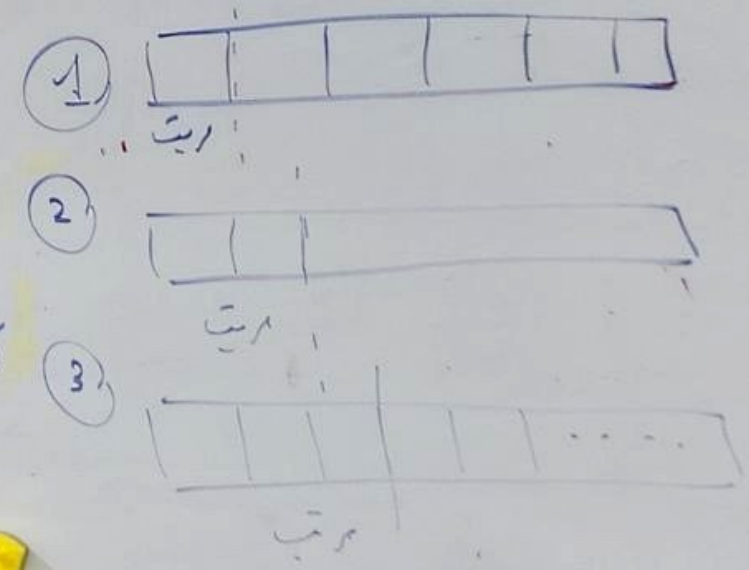
$2^{10} = 1000$

50 100

Bubble

sorting

هو عملية اعادة تنظيم عناصر مجموعة من البيانات وفق معيار محدد مثل ترتيب البيانات تنازليا او تصاعديا.



50 100 6 8 90 129 30 -

Bubble Sort

selection sort

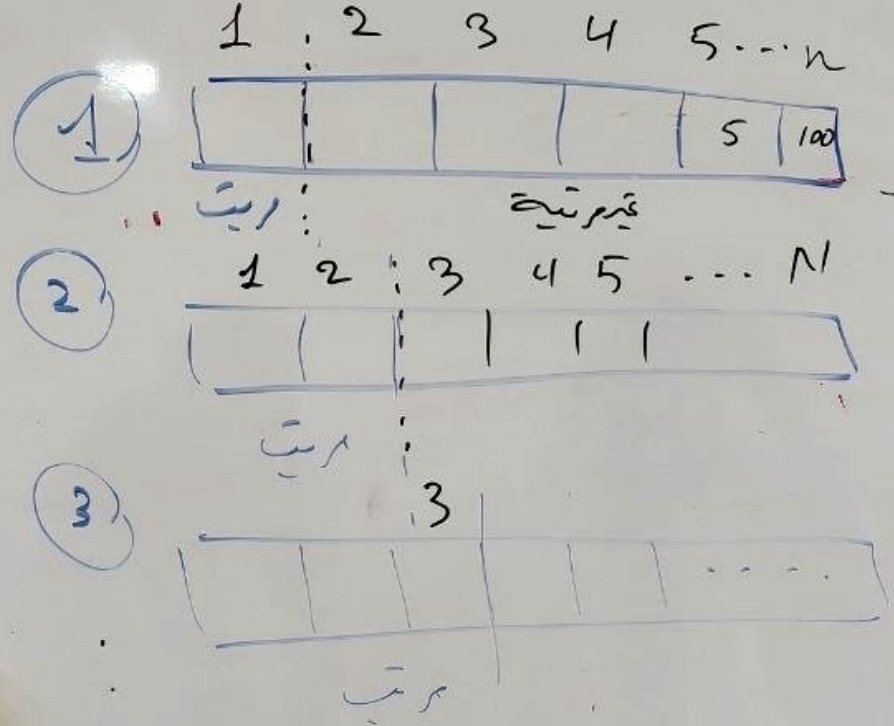
هو خوارزمية ترتيب بسيطة: تعتمد على اختيار أصغر عنصر من الجزء غير المرتب في القائمة (مصفوفة) في كل دورة ثم وضعه بموقعه الصحيح بداية القائمة. تتكرر هذه العملية لكبة ترتيب جميع عناصر القائمة.

```
for i = 1 to n-1
  for j = i+1 to n
```

50 100 6 8 90 120 30 5

min = x[1]

Bubble Sort



هي خوارزمية تعتمد على مقارنة كل عنصر مع العنصر الذي بعده وتبديلهما اذا كانا بترتيب خاطئ بحيث تطفو العناصر الأكبر تدريجياً الى نهاية القائمة (مصفوفة)

N

Selection Sort

خوارزمية ترتيب بسيطة تعتمد على إيجاد أصغر عنصر من الجزء غير المرتب من القائمة (مصفوفة) ثم وضعه في موقعه الصحيح في بداية القائمة. تتكرر هذه العملية حتى يتم ترتيب جميع العناصر.